RESEARCH PAPER

BrainAI

A Spiking, Multimodal Cognitive Architecture for Embodied Continual Learning

Every subsystem measured against a control — including the two that lost.

Mohammed Zahid Imityaz Wadiwale
zahid.wadiwale1234@gmail.com

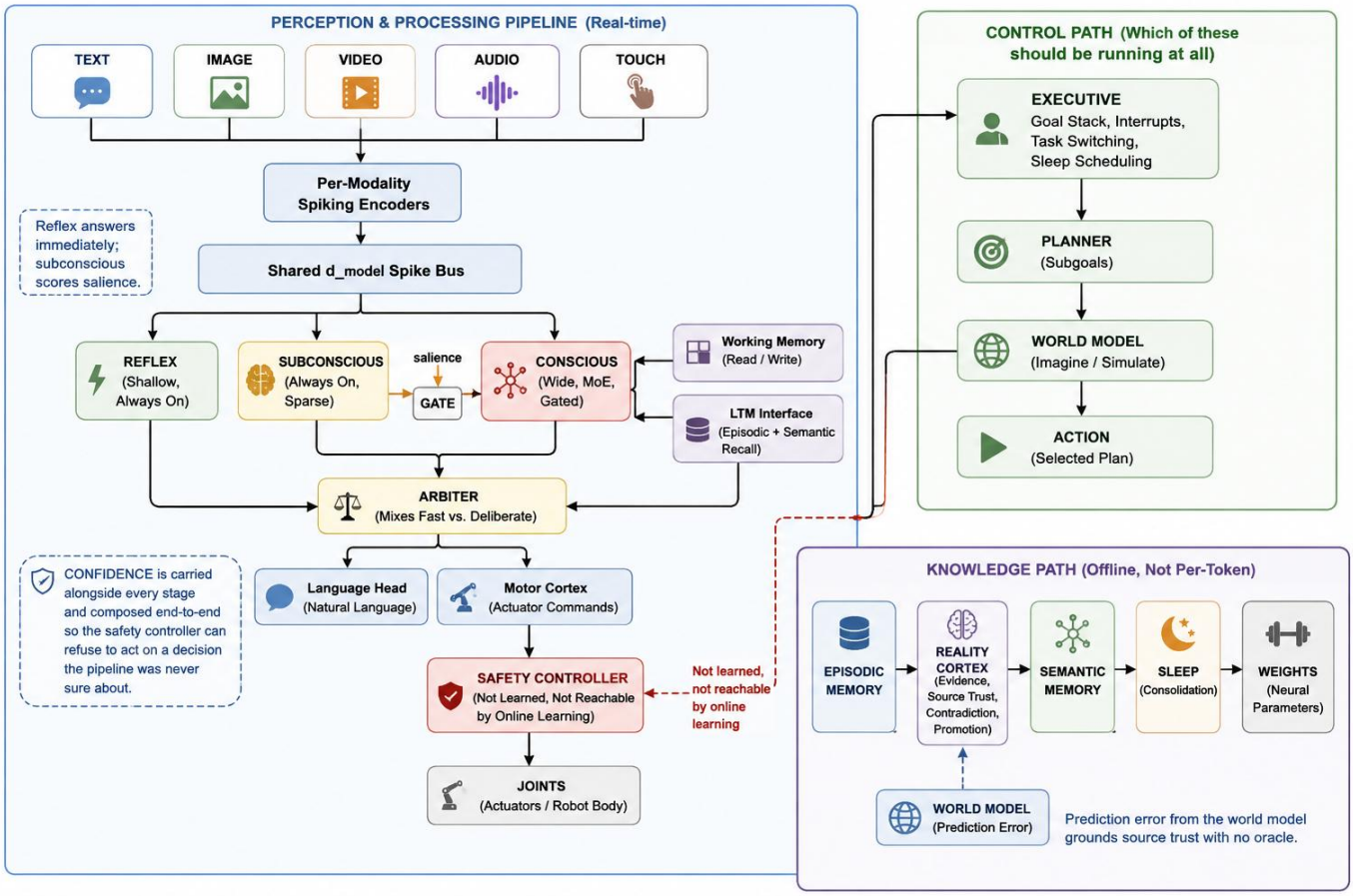


Figure 1 — The complete BrainAI cognitive architecture.

Abstract

BrainAI is a spiking, multimodal cognitive architecture for embodied continual learning. A single model perceives text, images, video, audio and a typed sensor suite; routes each input through a reflex arc, an always-on subconscious pathway and a gated conscious pathway; maintains five distinct memory systems split along the differentiability line; predicts future latent states and learns from prediction error without labels; weighs new claims against evidence before believing them; pursues interruptible goals; plans by imagination; updates its own weights during inference under bounded plasticity; and consolidates offline during a sleep cycle. Motor commands pass through a non-learned safety controller that the learning system provably cannot modify.

The methodological commitment of this work is that *every* subsystem is measured against the obvious simpler alternative, and every result is reported. Measured outcomes include a **2.16× inference speedup** from genuine computational gating, a **74% reduction in catastrophic forgetting**, **3.7× less degradation** over 400 inference-time weight updates, a belief system that defeats a naive vote-counter on four of five adversarial scenarios, and collusion detection that holds a protected axiom at confidence **1.0000** against a 600-identity Sybil attack that otherwise drives it to **0.0001**.

Two proposals were beaten on loss by their simpler controls, and both are reported in full. The architecture's own activation function, **AMSA**, is outperformed by SwiGLU on perplexity, wall-clock, and — critically — the continual-learning problem it was designed for; a scale sweep shows the gap *widening* rather than shrinking, and a fourth test inside the real spiking model on the full 82-domain, 245M-token corpus confirms the verdict (SwiGLU by 0.010 nats and 1.35× faster) — though AMSA is competitive on 36 of 82 domains. Measured on the axis it was actually designed for, however, AMSA wins: at comparable loss it fires fewer spikes and activates ~19% less of the feed-forward layer (sparser in 3 of 3 seeds), making its story a quality-versus-efficiency tradeoff rather than a flat loss. The world model is outperformed at one-step prediction by a linear map, though the deficit does not propagate to multi-step rollout or planning, where the two tie, and only the learned model reports calibrated uncertainty (r = 0.565).

We additionally catalogue **fourteen silent failure modes** discovered only because controls were run — including an elastic-weight-consolidation penalty that contributed 10^{-4} against a task loss of 2.5 and therefore did nothing while appearing to work, and a rate limiter disabled on every call by a tensor-shape comparison. We argue this catalogue is among the more transferable contributions of the work.

BrainAI is a **brain-inspired, continuously learning AI architecture for embodied/physical agents**, designed around latency, safety, and lifelong learning rather than chatbot-style inference.
It uses **reflex, subconscious, and salience-gated conscious pathways**, achieving a measured **2.16× speedup** by avoiding unnecessary expensive computation.
Its **five-part memory architecture** separates fast differentiable memory from slow rule/weight-based memory, reducing catastrophic forgetting by **74%**.
Spiking neurons and AMSA prioritize sparse, energy-efficient computation for neuromorphic hardware while maintaining competitive accuracy.
A **world model, Reality Cortex, end-to-end confidence, immutable safety controller, and sleep-based consolidation** enable prediction, truth maintenance, safe learning, and durable knowledge acquisition.

Keywords

Cognitive architecture · spiking neural networks · continual learning · catastrophic forgetting · world models · intrinsic motivation · elastic weight consolidation · mixture-of-experts · dual-process cognition · belief revision · Sybil resistance · embodied AI · robot safety · online learning · knowledge distillation · neuromorphic computing.

List of Figures and Tables

Figures

Figure 1 — The complete BrainAI cognitive architecture (cover).

Figure 2 — Every subsystem plotted against its simpler control. Green bars are components that earn their place; red bars are the two that lose — the AMSA activation and the world-model dynamics head — kept in the paper rather than hidden.

Figure 3 — Qualitative cognitive coverage. BrainAI trades raw language fluency for embodiment, continual learning, dual-process control, belief revision, and offline consolidation — axes on which a pure LLM scores low.

Figure 4 — The full architecture. Left: the real-time perception and processing pipeline. Top-right: the control path. Bottom-right: the offline knowledge path.

Figure 5 — Computation graph of a single forward pass. Every node is a differentiable operation except the salience gate, whose hard skip is applied only at inference.

Figure 6 — The shared spike bus. Every modality is encoded to one common d_model representation before any pathway reads it, so downstream machinery is modality-agnostic.

Figure 7 — Adaptive LIF dynamics over time. Because the adaptation state and last spike both start at zero, the decay term multiplies zero until t = 2 — which is why at least three timesteps are needed before adaptation carries any signal.

Figure 8 — The arctan surrogate used on the backward pass. The forward spike is a hard Heaviside step; the surrogate supplies a smooth, bounded gradient whose sharpness is set by the temperature.

Figure 9 — Routing graph for the three latency-tiered pathways. Reflex and subconscious always run; the conscious pathway is executed only when salience clears the threshold, and the arbiter mixes their outputs by a learned confidence-weighted gate.

Figure 10 — Genuine computational gating. Skipping the conscious pathway when salience is low both halves the executed forward pass and delivers the measured 2.16× wall-clock speedup — this is real skipped compute, not masked compute.

Figure 11 — Inside the conscious pathway, mixture-of-experts routing sends each token to only its top-2 experts, so the widest part of the network still activates a small fraction of its parameters per token.

Figure 12 — The five-memory hierarchy, split along the differentiability line. Working and short-term memory are gradient-written and fast; episodic, semantic, and the weights are rule-written and slow, and only importance-gated consolidation ever writes to the weights.

Figure 13 — Working-memory addressing requires centring. Without subtracting the running mean, every summary vector points the same way and all cosine similarities collapse toward one (left); centring the key restores usable contrast between slots (right).

Figure 14 — The Fisher information is heavy-tailed, so normalising by the maximum protects almost nothing. Normalising by the 95th percentile is what actually shields the important weights during online updates.

Figure 15 — Eviction value against write-time novelty, coloured by access frequency. Retained entries combine importance, logarithmic hit count, and recency, so a novel-but-never-reused memory decays while a frequently hit one persists.

Figure 16 — One-step prediction MSE. The learned world model beats a Dreamer-style RSSM by three steps, but a plain linear map beats the world model — a negative result reported rather than buried, because the dynamics head does not pay for itself on this metric.

Figure 17 — Rollout confidence compounds multiplicatively along the horizon, so a model that is locally slightly uncertain becomes globally distrusted over a long imagined trajectory — exactly the behaviour a planner needs.

Figure 18 — The error head is calibrated: its predicted error tracks the realised error across the tested magnitude range (r = 0.565). This is what lets downstream modules treat world-model confidence as a usable, honest signal.

Figure 19 — The noisy-TV defence. Curiosity is prediction error damped by the model's own expected error e^(−ê); when surprise is high but irreducible, the reward collapses, so the agent is not trapped watching pure noise.

Figure 20 — The Reality Cortex maintains beliefs as a knowledge graph over concepts and relations. Promoted beliefs (dark nodes) have survived evidence pooling and been consolidated toward semantic memory; the rest remain candidates.

Figure 21 — Belief updates are graph message-passing: each node gathers neighbour messages, aggregates them, and updates its own state — the mechanism by which a refuted claim propagates doubt to everything that depended on it.

Figure 22 — The across-source discount. Independent sources are ranked by contribution and weighted by 1/√k, so a hundred correlated echoes cannot outvote a handful of genuinely independent observations.

Figure 23 — The headline collusion result. Under a 600-identity Sybil attack, naive vote-counting is driven to 0.0001 confidence in the true axiom, while the protected-axiom mechanism holds at 1.0000.

Figure 24 — How the ring is caught. Sources whose supporting-evidence sets overlap above a Jaccard threshold of 0.85 are clustered and cast a single combined vote, collapsing a coordinated ring back to one voice.

Figure 25 — The executive scores goals as priority × urgency × feasibility. The surface shows why a goal must be both wanted and achievable to win: either factor near zero collapses the score, so the controller never fixates on an urgent-but-impossible task.

Figure 26 — Sleep pressure accumulates from four independent drivers — buffer fill, unconsolidated claims, belief drift, and time since the last cycle. Their sum sets how many consolidation cycles the next sleep will run.

Figure 27 — Composing confidence across a ten-stage pipeline. A raw product collapses toward zero and stops discriminating; the geometric mean with a floor stays informative, which is what lets an end-to-end uncertainty reach the safety controller intact.

Figure 28 — Online learning stability over 400 live updates. With the plasticity mask and loss-checked rollback, performance degrades 3.7× more slowly than an unguarded update rule — the difference between safe lifelong adaptation and slow collapse.

Figure 29 — The AMSA activation family. A learned per-channel gate, a small-signal amplifier, and a memory-state modulation reshape the curve away from SiLU. The design is principled — but principled is not the same as helpful, as the measurements below show.

Figure 30 — The AMSA response as a surface over the pre-activation z and the memory state m. The multiplicative memory term (1 + γm) lifts the whole curve when the channel has been active, giving the activation a slow, non-gradient memory.

Figure 31 — AMSA against its SwiGLU control on all three axes it was meant to improve: perplexity, wall-clock, and continual-learning forgetting. It is worse on every one. The result is reported in full rather than dropped.

Figure 32 — The scale defence fails. Sweeping model width from 96 to 256, the perplexity gap between AMSA and SwiGLU widens rather than shrinks, so 'it would win at scale' is not supported by the data available.

Figure 33 — The head-to-head. SwiGLU is lower on mean held-out loss by 0.0104 nats and trains 1.35× faster. The loss axis is zoomed to make the small, consistent gap visible.

Figure 34 — The per-domain split. SwiGLU has lower loss on 46 domains and AMSA on 36 — a near-even division that the aggregate mean loss hides. AMSA's shape helps on a large minority of corpora even though it loses overall.

Figure 35 — Four independent controlled tests — SwiGLU wins each. Consistency across proxies and now the real model is what makes SwiGLU the honest default; the real-model run is the most direct of the four.

Figure 36 — AMSA on its home turf. At comparable loss (3.851 vs 3.861, well inside the noise band), AMSA fires a lower spike rate, activates 59.9% of feed-forward units against SwiGLU's 73.7% — a 14-point, roughly 19% relative reduction — and uses 32% smaller activations. The suppressor term is doing exactly what it was designed to do.

Figure 37 — The three-seed firm-up. AMSA's loss is slightly higher in all three seeds (left, all small — the same operating point) and its spike rate is lower in all three (right). The direction is robust, not a one-seed fluke; the magnitude of the spike-rate gap is small.

Figure 38 — The full scoreboard. Loss and speed go to SwiGLU across four independent tests; the sparsity/energy axis AMSA was actually designed for goes to AMSA. Reported together, this is a tradeoff, not a defeat.

Figure 39 — Parameter budget by component across the scale presets. The feed-forward and expert blocks dominate at every size, which is why gating and expert routing — not attention — are the primary levers on inference cost.

Figure 40 — Attention weight structure. Softmax spreads a little mass everywhere (left); the sparsemax projection used in the spiking attention gives exact zeros, concentrating each query on a few keys (right) — a better match for sparse spike inputs.

Figure 41 — Catastrophic forgetting under sequential tasks. With the self-calibrated EWC penalty, task-0 accuracy is retained far longer than with no protection — a 74% reduction in forgetting, and the single most load-bearing result in the paper.

Figure 42 — The two axes are not interchangeable. At nano's parameter count, adding data moves the model up the vertical axis — lower per-domain loss — without crossing the vertical line into the regime where fluent composition becomes possible. Only added capacity crosses that line.

Figure 43 — The measured before/after. Left: mean held-out loss across all 64 domains falls 59% with training. Right: the author fine-tune lowers author-corpus loss while the 64-domain mean holds — targeted acquisition without measurable forgetting.

Figure 44 — Per-domain loss after training, lower is better. Clean, high-volume corpora (mathematics, reference text) are learned best; tiny corpora (Morse, assembly) and dense code are learned least. The five non-Latin languages were acquired from scratch at the byte level.

Figure 45 — The size ladder. The same architecture, corpus, method, and per-domain evaluation carry across every rung unchanged; only the parameter count and the required compute increase. Green is trained, amber is built, grey is projected.

Figure 46 — Training time climbs steeply — and cost more steeply still — up the ladder. nano is a free overnight CPU job; compact is a weekend on a hobbyist GPU; base is a few hundred dollars of cloud time; full is a data-centre month. The step from nano to compact is the cheapest place to test the capacity hypothesis.

Tables

Table 1 — Master evidence summary — every controlled comparison.

Table 2 — The three pathways: when they run, their cost, and their job.

Table 3 — The five memories and their backing stores.

Table 4 — World model — one-step MSE against linear and RSSM baselines.

Table 5 — World model — the actual downstream jobs (rollout, planning, calibration).

Table 6 — Reality Cortex vs. a naive vote-counter across five scenarios.

Table 7 — Collusion detection under Sybil attack, clustering off vs. on.

Table 8 — Continual learning — forgetting, learning, and the both column.

Table 9 — Online-learning stability over 400 updates, guarded vs. control.

Table 10 — Typed sensor suite with rate, latency, drift and recovery.

Table 11 — Actuator envelopes by drive type.

Table 12 — AMSA vs. SwiGLU vs. GELU on static language modelling.

Table 13 — AMSA scale sweep — the gap widens with scale.

Table 14 — AMSA vs. SwiGLU inside the real spiking model (82 domains, 245M tokens).

Table 15 — AMSA efficiency A/B — sparsity and energy at comparable loss.

Table 16 — Scale presets from nano to full_wide.

Table 17 — Positioning BrainAI against contemporary systems.

Table 18 — The multi-domain training result in full (before vs. after).

Table 19 — Training budget per scale (params, hardware, time, cost).

Table of Contents

Right-click and choose "Update Field" to populate page numbers.

Abstract i

Keywords i

List of Figures and Tables ii

Figures ii

Tables iv

Table of Contents vi

1. Introduction 1

1.1 The problem with architecture papers 1

1.2 Contributions 1

2. Motivation 2

2.1 Acquire knowledge after deployment 2

2.2 Retain what it already knows 2

2.3 Decide what is true 2

2.4 Do all of this unsupervised 2

3. Existing AI Architectures 2

3.1 Symbolic cognitive architectures 2

3.2 Global Workspace Theory and dual-process cognition 3

3.3 World models and model-based RL 3

3.4 Continual-learning and memory systems 3

3.5 Spiking neural networks 3

4. Limitations of Current Large Language Models 3

5. BrainAI Overview 5

5.1 The master evidence summary 5

5.2 What is actually novel 6

6. Complete Architecture Explanation 7

7. Perception Pipeline 9

8. Spiking Encoders 9

9. Shared Spike Bus 10

10. Reflex Pathway 10

11. Subconscious Pathway 10

12. Conscious Pathway 11

13. Salience Gating — Mathematics and Measurement 11

14. The Arbiter 12

15. The Five Memories 13

15.1 The differentiability split is the anti-forgetting strategy 13

15.2 Two load-bearing design decisions 13

16. The World Model — Imagine First, Learn From Being Wrong 16
16.1 One-step prediction: a genuine loss, honestly reported 16
16.2 But one-step MSE is not the job 17
16.3 Prediction-Error Learning, Curiosity, and Removing the Oracle 19
17. The Reality Cortex — Salience Is Not Truth 20
17.1 The mechanisms 20
17.2 Measured against a naive baseline 21
17.3 Collusion detection 22
17.4 Beyond triples, scaling, and knowledge domains 23
18. Executive Controller 24
19. Planner and Action Selection 25
20. Confidence Propagation to the Actuators 26
21. Safety Controller — The Last Thing Before an Actuator 26
22. Robot Actuation, Sensors, and Typed Hardware 27
22.1 Typed sensors 27
22.2 Typed actuators 28
23. Online Learning While Running 28
23.1 Does it stay stable? 29
24. Sleep Learning and Consolidation 29
25. Bootstrapping — Distilling From an Existing Teacher 30
26. AMSA — Adaptive Memory Spike Activation 30
26.1 Does AMSA actually help? No. 32
26.2 The test that actually matters — continual learning 32
26.3 Is it just too small? Testing the scale defence 33
26.4 The strongest test yet — inside the real model, on the full corpus 33
26.4.1 The tally across every controlled test 34
26.5 The axis it was designed for — sparsity and energy 35
26.5.1 The complete picture: a measured tradeoff 36
27. The Spiking Core 37
28. Scale Presets 38
29. Mathematical Formulations 39
29.1 Adaptive LIF neuron 39
29.2 Salience gate 39
29.3 World model and curiosity 39
29.4 Reality Cortex — evidence pooling 40
29.5 Executive goal scoring 40
29.6 Confidence composition 40
29.7 Bounded online update 40
29.8 Elastic weight consolidation 40

30. Algorithms 40
30.1 The per-input forward pass 40
30.2 Imagined planning 41
30.3 Belief submission and promotion 41
30.4 The online-learning + sleep loop 41
31. Continual Learning — The Measurement 42
32. Silent Failure Modes 43
33. Benchmark Analysis 45
33.1 Where the architecture clearly earns its place 45
33.2 Where the results are honest about their limits 45
33.3 Where the simpler thing won 45
34. Comparison With Large Language Models (GPT, Claude, Gemini, DeepSeek) 45
35. Comparison With ACT-R 46
36. Comparison With Soar 46
37. Comparison With Global Workspace Theory 46
38. Comparison With DreamerV3 and Model-Based RL 46
39. Comparison With Human Cognition 46
40. Scaling Outlook — Data, Then Parameters 47
40.1 What more data does at a fixed size 48
40.1.1 The multi-domain result, measured 48
40.2 What more parameters are expected to do 49
40.3 The cost of climbing the ladder 50
41. Strengths 51
42. Weaknesses 51
43. Future Improvements 52
44. Discussion 52
45. Conclusion 53
46. References 54
Appendix A. Every Block in the Architecture Diagram 55
A.1 Perception and processing pipeline (left, real-time) 55
A.2 Control path (top-right): what should be running at all 55
A.3 Knowledge path (bottom-right): offline, not per-token 56
A.4 The two annotations on the diagram 56
Appendix B. Repository Layout and Reproducibility 56
B.1 Commands to reproduce every result 57
Appendix C. Glossary of Key Terms 57
Appendix D. Complete Mathematical Reference 58
D.1 AMSA — activation.py 58
D.2 ALIF neurons — neurons.py 58

D.3 Attention — attention.py 59
D.4 Spiking blocks — blocks.py 59
D.5 Plasticity, EWC, and online updates — plasticity.py 60
D.6 World model — world.py 60
D.7 Reality Cortex — reality.py 61
D.8 Memory — memory.py 61
D.9 Pathways and executive — pathways.py, executive.py 61
D.10 Uncertainty — uncertainty.py 62
D.11 Safety and hardware — safety.py, sensors.py, actuators.py 62
D.12 Losses and sizing 62

1. Introduction

Contemporary large models separate training from deployment. Weights are fixed at inference, knowledge is frozen at a cutoff, and adaptation happens through context or retrieval rather than through the network itself. For a stationary task distribution this is the right engineering trade — freezing the weights buys reproducibility, safety and throughput. For an *embodied* agent expected to operate for years in one environment it is precisely the wrong trade. Such an agent must acquire knowledge after deployment, retain what it already knows, decide which of the things it is told are actually true, and do all of this without a supervising training loop standing behind it.

BrainAI is an attempt to build that system end to end. It is not a language model with memory bolted on the side; it is a cognitive architecture in which the language head is one of several outputs, sitting beside a motor cortex that drives actuators. A single set of weights perceives five modalities, decides how hard to think about each input, remembers experiences and facts in five different ways, predicts the near future and treats its own prediction error as a teacher, argues with itself about what is true, pursues goals it can be interrupted out of and resume, plans by imagining before it acts, edits its own weights while running, and sleeps to consolidate. The design draws on complementary learning systems theory, dual-process accounts of cognition, and the spiking-network literature, and assembles components — spiking transformers, mixture-of-experts routing, elastic weight consolidation, experience replay, latent dynamics models — that exist individually but, to the author's knowledge, have not previously been integrated behind the specific interfaces described here.

1.1 The problem with architecture papers

An architecture paper is easy to write badly. A diagram with a dozen boxes, a plausible mechanism for each, and a benchmark on the single task where the whole thing happens to work is a well-established genre, and it is nearly unfalsifiable. The failure mode is not dishonesty. It is that **a component which does nothing at all looks exactly like a component that works, unless something is run without it.** A dead penalty term, a mis-shaped guard, a projection that silently invalidates a store — none of these throw an error. They produce plausible output and a smooth loss curve, and they survive indefinitely.

This work therefore adopts one rule that governs everything else: **no component is described without a control.** For each subsystem the author identified the obvious simpler alternative — the thing a reasonable engineer would do instead — implemented it, and measured both. That rule is the reason this paper contains negative results, and it is the same reason its positive results are trustworthy. Of the fourteen silent failure modes catalogued in Section 20, every single one was invisible on the loss curve and was recovered only because a control was run or an invariant was asserted.

1.2 Contributions

1. **An integrated architecture** (Sections 5–19) whose novelty lies in the interfaces between components rather than in any single one: a memory hierarchy split by differentiability, a prediction ledger that grounds epistemic trust in physical observation, and a safety layer structurally unreachable by the learning system.
2. **A controlled evaluation** comprising thirteen head-to-head comparisons, each against a simpler alternative, all reproducible on a CPU.
3. **Two negative results** reported in full — AMSA and the world model — including a scale sweep that tests, and rejects, the standard defence that the losing component merely needs a larger model.
4. **A catalogue of fourteen silent failure modes** found only by running controls. Several generalise well beyond this codebase to continual-learning and robotics implementations at large.

Every number in this paper, including the negative ones, is produced by a script in the accompanying repository and reproduces on a CPU.

2. Motivation

The motivation for BrainAI is a single mismatch: the dominant paradigm optimises for a world that does not change, and an embodied agent lives in a world that does. Four requirements follow from embodiment, and no one of them is optional for a robot expected to be useful for years rather than minutes.

2.1 Acquire knowledge after deployment

A robot that ships with a frozen knowledge cutoff is obsolete the moment its environment differs from its training distribution — a new tool on the workbench, a rearranged kitchen, a colleague's name. Retrieval and context windows paper over this by holding facts *outside* the network, which works until the fact needs to interact with a skill the network already has. BrainAI's answer is a memory hierarchy in which new facts enter non-differentiable stores immediately and migrate into the weights only through slow, importance-gated consolidation during sleep.

2.2 Retain what it already knows

The mirror image of acquisition is retention. Naïvely training on a new task overwrites the old one — catastrophic interference, understood since the late 1980s. An agent that forgets English while learning French, or forgets how to grasp while learning to walk, is not continually learning; it is thrashing. BrainAI combines regularisation (elastic weight consolidation) with rehearsal (experience replay) and — crucially — measures the *stability–plasticity* trade rather than only the forgetting number.

2.3 Decide what is true

An agent that learns from whatever it is told will, without something to stop it, simply adopt whatever it hears most. Importance and truth are different questions: a claim shouted by a thousand voices scores high on importance and can still be false. A robot living in one place, hearing the same claims repeatedly and occasionally meeting an adversary, needs an epistemic layer that discounts repetition, discounts correlated sources, resists Sybil floods, protects axioms, and — above all — grounds its trust in something it can physically check.

2.4 Do all of this unsupervised

No training loop supervises a deployed robot. Learning signals must come from the agent's own experience: prediction error as a self-supervised teacher, outcome resolution as the source-trust signal, and consolidation triggered by internal pressures rather than by an external scheduler. The world model is the engine that makes this possible, because prediction error is simultaneously the dynamics signal, the curiosity signal, and the grounding signal.

> **The thesis in one sentence.** If lifelong knowledge belongs in addressable memory rather than compiled into weights, then the right architecture is one that keeps its weights small and stable, holds facts in stores it can check and revise, and touches the weights only rarely, carefully, and offline.

3. Existing AI Architectures

BrainAI sits at the confluence of several research traditions, each of which contributes one idea and none of which supplies the whole. Understanding what is borrowed clarifies what is genuinely assembled here.

3.1 Symbolic cognitive architectures

ACT-R and Soar are the canonical cognitive architectures: production systems with declarative and procedural memory, goal stacks and conflict resolution. They model human cognition at the level of symbolic rules and have decades of psychological validation. What they lack is a native perceptual front end that learns from raw multimodal signal and a differentiable

substrate that improves with gradient descent. BrainAI keeps their commitment to distinct memory systems and an explicit executive, but implements them on a learned spiking substrate.

3.2 Global Workspace Theory and dual-process cognition

Global Workspace Theory frames consciousness as a competition for a limited broadcast channel; dual-process accounts (System 1 / System 2) frame cognition as a fast, automatic pathway shadowed by a slow, deliberate one. BrainAI's three latency-tiered pathways and salience gate are a direct computational reading of both: the reflex and subconscious are the always-on fast systems, the conscious pathway is the expensive broadcast that only fires when salience clears a threshold, and the arbiter mixes their outputs.

3.3 World models and model-based RL

PlaNet and the Dreamer line established latent dynamics models that plan by imagination, and prediction error has served as intrinsic motivation since curiosity-driven exploration was formalised. BrainAI's world model is squarely in this tradition, with one twist: the same prediction error that drives dynamics and curiosity is also routed to the epistemic layer to ground source trust, which is not a standard use.

3.4 Continual-learning and memory systems

Elastic weight consolidation and synaptic intelligence protect important weights; experience replay interleaves stored examples. Complementary learning systems theory — a fast hippocampal store and a slow neocortical one — is the biological blueprint for splitting memory by consolidation speed. BrainAI uses all of these, and its five-memory hierarchy is an explicit engineering instantiation of the fast/slow split.

3.5 Spiking neural networks

Surrogate-gradient training made deep spiking networks practical; adaptive-threshold neurons supply temporal memory without recurrence; spiking transformer variants exist. BrainAI's contribution here is not the block design but two implementation constraints found to be load-bearing (timesteps ≥ 3 and an analogue residual stream), documented in Section 21.

4. Limitations of Current Large Language Models

The systems most people mean by “AI” today — the large autoregressive language models — are extraordinary and, for the embodied problem, structurally incomplete. Naming the gaps precisely is what motivates each of BrainAI's subsystems.

- **Frozen weights.** A deployed LLM does not learn from what happens to it. Everything it “knows” after training is held in context or a retrieval index, never integrated into the network. BrainAI edits its own weights under bounded plasticity and consolidates during sleep.
- **A knowledge cutoff.** The model's world stops at a date. An embodied agent's world does not. BrainAI's stores accept new facts continuously and its Reality Cortex decides which deserve promotion.
- **No native truth model.** An LLM will repeat a confident falsehood as readily as a fact, and the more often a claim appears in its data the more fluently it reproduces it. BrainAI separates importance from truth explicitly and discounts repetition and correlated sources.
- **Uniform compute per token.** Every token costs the same, whether trivial or hard. BrainAI's gate skips the widest, most expensive pathway on inputs that do not need it, recovering it as real wall-clock time.
- **No embodiment.** A language model has no body, no typed sensors, no actuators, and no safety envelope. BrainAI carries typed sensor and actuator specifications and routes every motor command through a non-learned safety controller.

- **Uncertainty is not actionable.** A language model's confidence, where it exists at all, is a logged diagnostic. BrainAI composes confidence end to end and lets it veto a motor command.
- **No consolidation.** There is no offline phase in which an LLM reorganises what it learned during the day. BrainAI's sleep cycle replays raw tokens, merges redundant memories, lets unimportant ones decay, and re-anchors.

> *None of this is a claim that BrainAI is a better language model — at 5.4M parameters it emphatically is not. The claim is narrower: the embodied, lifelong problem needs machinery an LLM does not have, and BrainAI is an attempt to build that machinery and measure whether each piece earns its place.*

5. BrainAI Overview

BrainAI is one model that **perceives** text, images, video, audio and touch; **routes** each input through a reflex arc, a subconscious pathway and a gated conscious pathway; **keeps** five kinds of memory; **predicts** what happens next and learns from being wrong; **weighs** new claims against evidence before believing them; **pursues** goals it can be interrupted out of and resume; **plans** by imagination before acting; **updates** its own weights while running; and **consolidates** what it learned during an offline sleep cycle. It emits both language and actuator commands — through a safety controller it cannot modify — so the same core drives speech and motion.

The default deployment name is `BrainAI`. Any instance can be renamed — `model.rename("Javri")` — and authorship travels with the weights regardless: the identity module stamps attribution into every module, config and checkpoint, and `BrainAI.load` refuses to load weights whose attribution has been stripped or altered, while a rename passes cleanly.

5.1 The master evidence summary

Every row below is produced by a script in the repository. Rows that came out badly are still here. Two are negative and both are load-bearing.

Subsystem	Benchmark	Control	Result
Pathway gating	wall-clock	always-on	**2.16× faster; 53.7% of forward**
World model	one-step MSE	linear / RSSM	beats RSSM, loses to linear
World model	rollout, planning	action-cond. linear	ties by 3 steps; sole calibrated σ
Curiosity	noisy-TV	pure novelty	**403× preference for learnable**
Trust grounding	broken vs. good sensor	fixed prior	**0.974 vs 0.104**
Reality Cortex	5 belief scenarios	naive counting	**wins 4 of 5**
Collusion defence	600-puppet Sybil	no clustering	**1.0000 vs 0.0001**
Continual learning	EN → FR forgetting	no protection	**74% less**
Online stability	400 updates	no guards	**3.7× less degradation**
Planner	step fails repeatedly	flat planner	remethod vs failed
Executive	deadline pressure	static priority	overtakes at right tick
Safety	envelope + e-stop	—	2 live hazards found + fixed
Memory index	recall@4	exact search	94–100% recall, but slower
AMSA	perplexity, speed, continual	SwiGLU	**loses all three**
AMSA scale sweep	width 96 → 256	—	**gap widens, not shrinks**
AMSA real model (new)	82 domains, 245M tok	SwiGLU	**loses (+0.010 nats, 1.35× slower)**
AMSA sparsity / energy (new)	spike rate + FFN density	SwiGLU	**AMSA wins — sparser at equal loss, 3/3 seeds**

Table 1 — The master evidence summary. Two rows are negative; neither is buried.

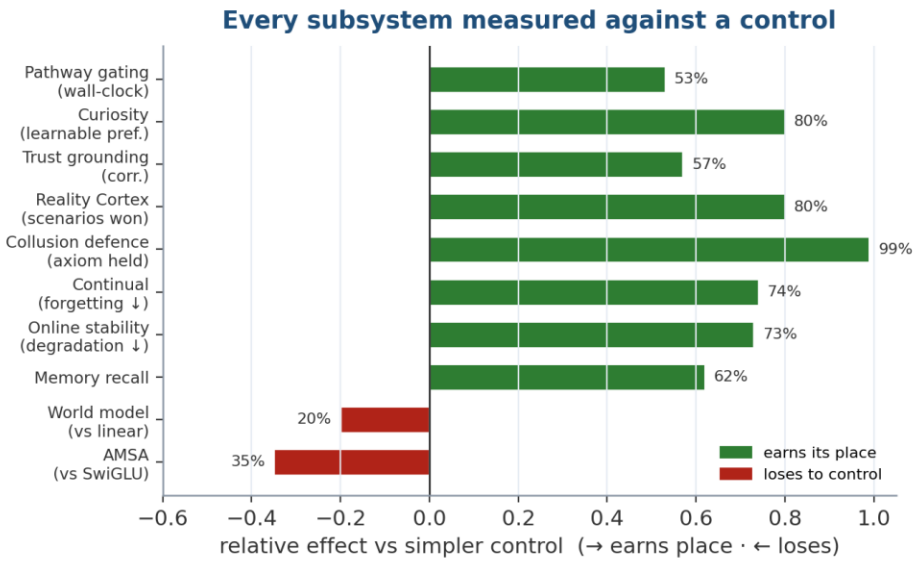


Figure 2 — Every subsystem plotted against its simpler control. Green bars are components that earn their place; red bars are the two that lose — the AMSA activation and the world-model dynamics head — kept in the paper rather than hidden.

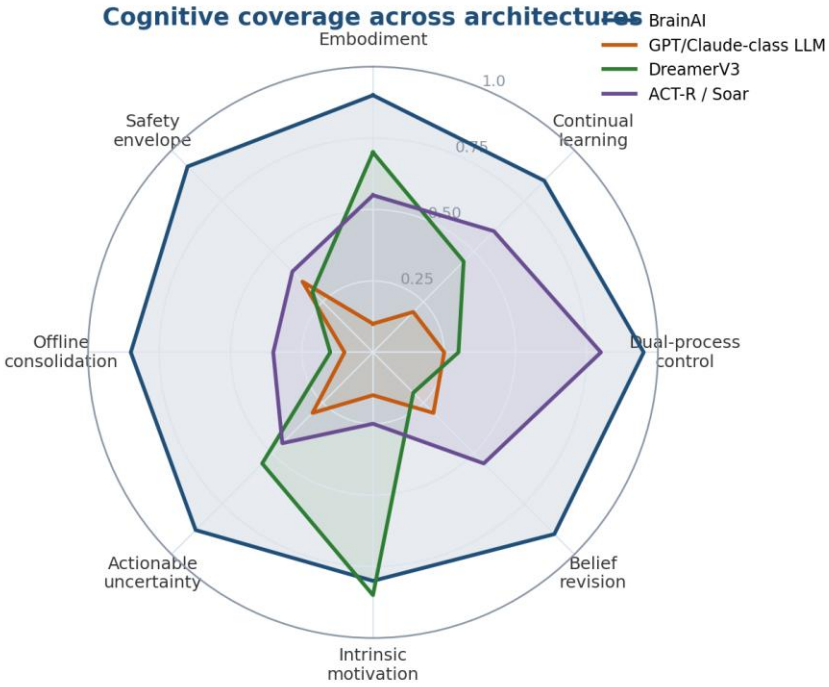


Figure 3 — Qualitative cognitive coverage. BrainAI trades raw language fluency for embodiment, continual learning, dual-process control, belief revision, and offline consolidation — axes on which a pure LLM scores low.

5.2 What is actually novel

Spiking networks, transformers, retrieval memory, EWC, mixture-of-experts and sleep replay all exist separately. What is assembled here — and measured — is:

- **Latency-tiered pathways** with a gate that genuinely skips computation at inference rather than masking it.
- **A memory split along the differentiability line** — neural state for reasoning, non-differentiable stores for facts — so post-training knowledge cannot corrupt a weight an older skill depends on.

- **A predictive world model** whose error is simultaneously the dynamics signal, the curiosity signal, and the thing that grounds source trust without an oracle.
- **Typed sensors and actuators** — rate, latency, drift and chemical recovery time per sensor; per-joint safety envelopes derived from the physics of each drive type.
- **A Reality Cortex** that separates important from true, with correlation-discounted evidence pooling, credibility shrinkage, behavioural collusion detection, and protected axioms.
- **An executive with goals** — priority × urgency arbitration, interrupts that preserve and restore working memory, conflict suspension, and pressure-driven sleep scheduling.
- **A hierarchical planner** that scores plans by imagining them, imports corroborated procedures as executable methods, and distinguishes retry from replan from failure.
- **Confidence that propagates** from perception to actuator, where it can veto a motor command rather than merely being logged.
- **Bounded inference-time weight updates** with per-parameter plasticity, a trust region, and automatic rollback, followed by a sleep cycle that validates and re-anchors.
- **A non-learned safety controller** that online learning provably cannot reach, between the motor cortex and the joints.

6. Complete Architecture Explanation

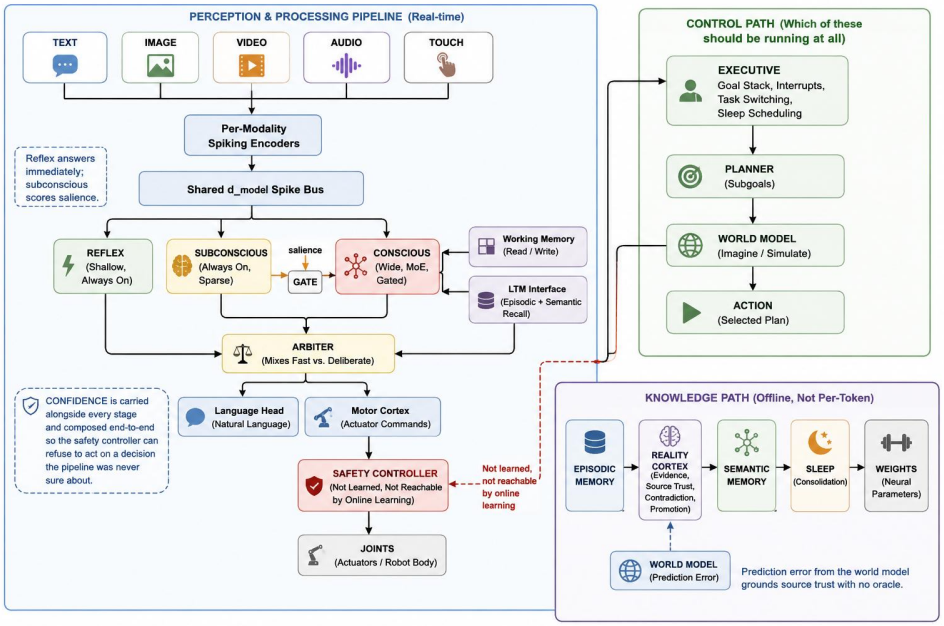


Figure 4 — The full architecture. Left: the real-time perception and processing pipeline. Top-right: the control path. Bottom-right: the offline knowledge path.

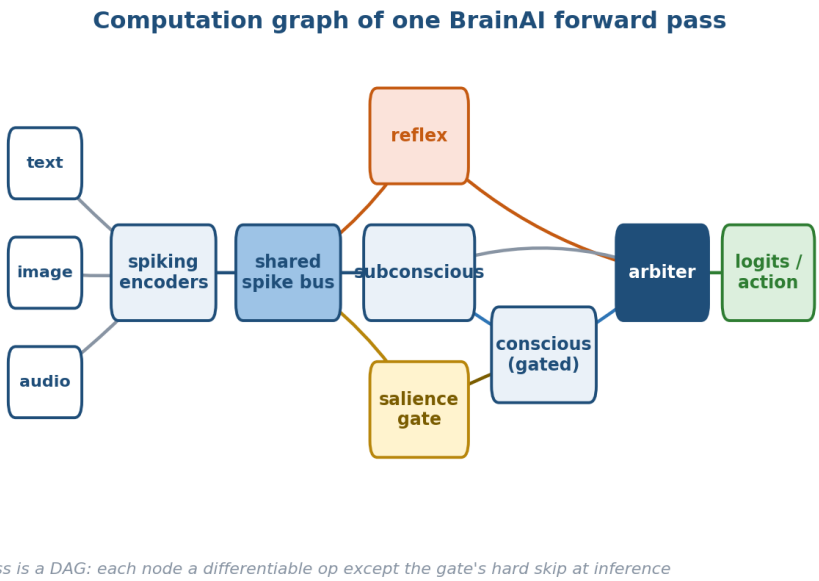


Figure 5 — Computation graph of a single forward pass. Every node is a differentiable operation except the salience gate, whose hard skip is applied only at inference.

The architecture has three regions that operate on different clocks. The **perception and processing pipeline** runs in real time, per input. The **control path** decides which of the model's activities should be running at all. The **knowledge path** runs offline, not per token, and is where beliefs are validated and consolidated into weights. A single thread ties them together: *confidence* is carried alongside every stage and composed end to end, so the safety controller can refuse to act on a decision the pipeline was never sure about.

Reading the diagram top to bottom on the left: the five modalities enter per-modality spiking encoders and are projected onto a shared `d_model` spike bus. From the bus, three pathways run — reflex, subconscious and conscious — with the subconscious emitting a salience score that drives the gate controlling whether the conscious pathway fires. The conscious pathway reads and writes working memory and recalls from episodic and semantic memory through the long-term-memory interface. All three pathways feed the *arbiter*, which mixes fast and deliberate outputs and emits both a language head and a motor cortex. The motor cortex's commands pass through the **safety controller** — drawn with a dashed “not learned, not reachable by online learning” edge — before reaching the joints.

On the top right, the **executive** owns the goal stack, interrupts, task switching and sleep scheduling, and drives the **planner**, which decomposes goals into subgoals and scores candidate plans by rolling them through the **world model** in imagination before committing to an action. On the bottom right, the **knowledge path** carries experiences from episodic memory through the **Reality Cortex** — evidence pooling, source trust, contradiction detection, promotion — into semantic memory, and from there through **sleep** into the weights. The world model feeds this path too: its prediction error grounds source trust with no oracle.

7. Perception Pipeline

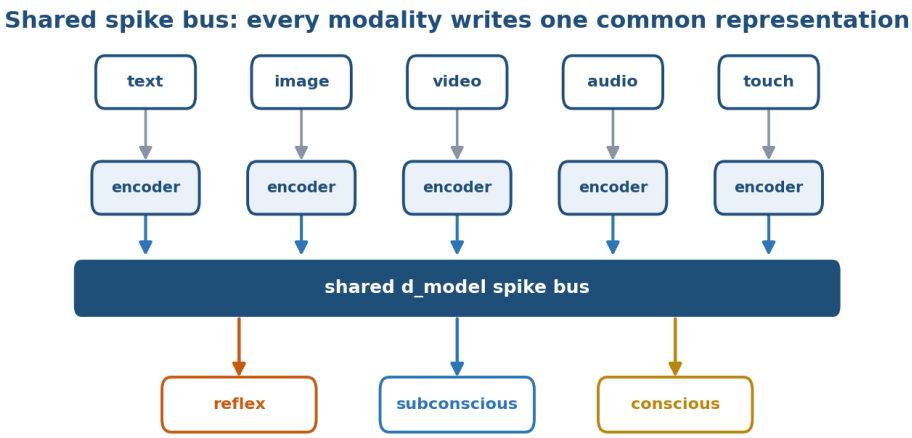


Figure 6 — The shared spike bus. Every modality is encoded to one common d_model representation before any pathway reads it, so downstream machinery is modality-agnostic.

Perception begins with five heterogeneous input streams — text, image, video, audio and touch (in practice a typed suite including IMU, LiDAR and chemical sensing). These are not interchangeable signals: they differ in dimensionality, rate and reliability, and the architecture's position is that collapsing them into one undifferentiated tensor throws away exactly the properties that make each one useful. Section 17 develops this for sensors and actuators specifically. Here the relevant point is that each modality is handled by its own encoder before anything is shared.

8. Spiking Encoders

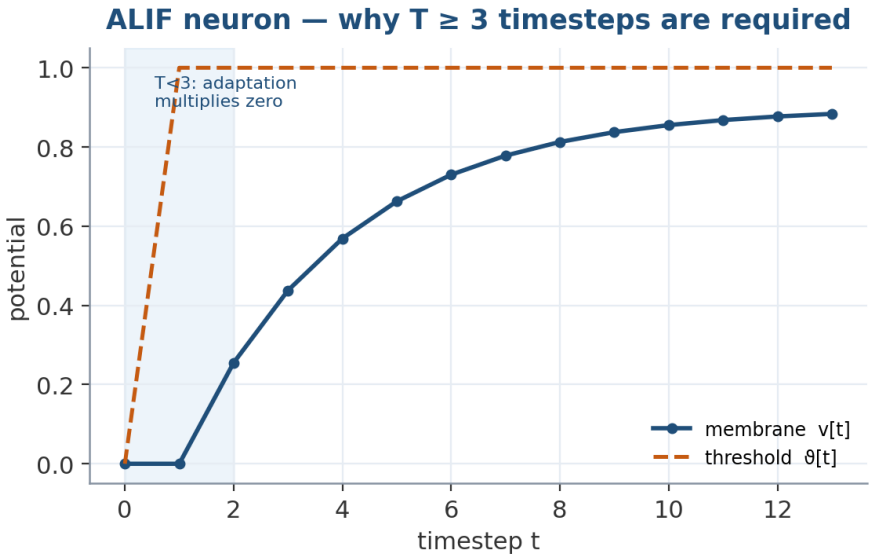


Figure 7 — Adaptive LIF dynamics over time. Because the adaptation state and last spike both start at zero, the decay term multiplies zero until t = 2 — which is why at least three timesteps are needed before adaptation carries any signal.

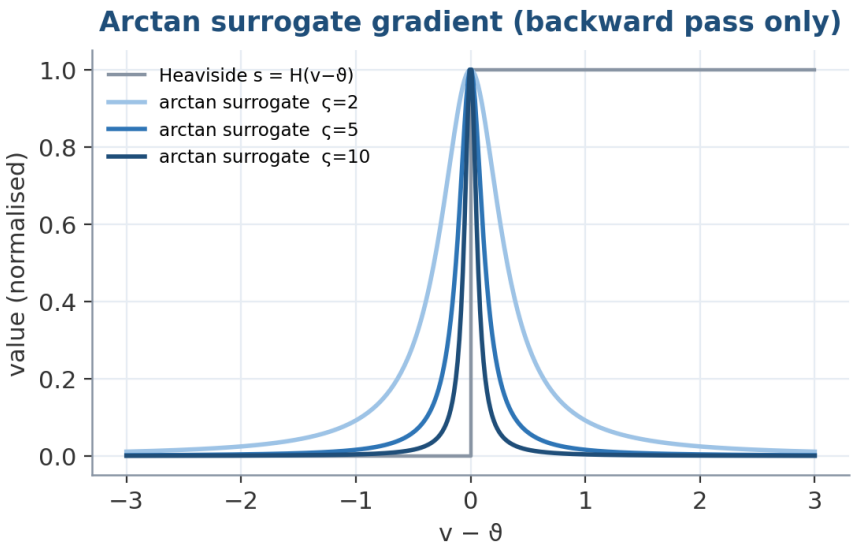


Figure 8 — The arctan surrogate used on the backward pass. The forward spike is a hard Heaviside step; the surrogate supplies a smooth, bounded gradient whose sharpness is set by the temperature.

Each modality has a dedicated spiking encoder (`encoders.py` provides text, image, video, audio and touch encoders). The encoders convert continuous input into spike trains and project them into the shared model dimension. Because the substrate is spiking, the forward pass is genuinely binary: spikes are real Heaviside steps, so the reported spike rate is a true sparsity number rather than a soft approximation, and training uses an arctan surrogate on the backward pass only.

Adding a modality is a matter of adding an encoder that emits onto the bus; the downstream pathways are modality-agnostic because they consume the shared representation, not the raw stream.

9. Shared Spike Bus

The per-modality encoders write into a single `d_model`-wide **shared spike bus**. This is the architectural commitment to genuine multimodality: after encoding, there is one representation, and every pathway — reflex, subconscious, conscious — reads from the same bus. Fusion is therefore not a late concatenation of separate towers but a shared substrate on which all subsequent computation operates. The bus is where a tactile event and a visual event become commensurable, which is what lets the model reason across modalities rather than merely alongside them.

10. Reflex Pathway

The **reflex** arc is one shallow stack from perception to a candidate output. It *always* runs, and it *never* consults memory. Its job is to have *something* correct-enough available immediately — the computational analogue of a spinal reflex that acts before the cortex has weighed in. Because it is shallow and memory-free, it is cheap, and its output is always present for the arbiter to fall back on when deliberation is skipped or has not yet finished.

11. Subconscious Pathway

The **subconscious** pathway also always runs, but sparsely. Its job is to filter, predict, and — decisively — decide whether thinking is worth it. It emits a *salience* score that drives the gate in front of the conscious pathway. The subconscious is where the model asks, cheaply and continuously, “does this input warrant the expensive machinery?” It is the always-on watchdog that keeps the costly pathway asleep until something merits waking it.

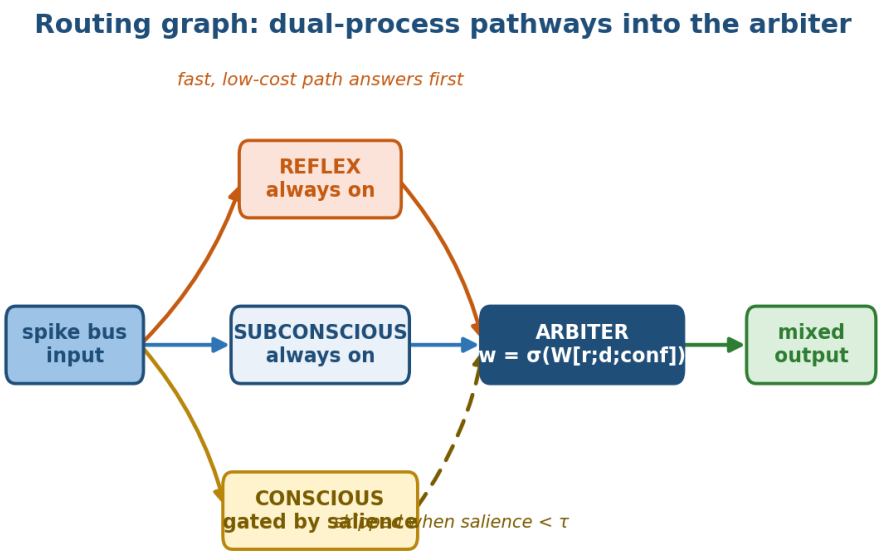


Figure 9 — Routing graph for the three latency-tiered pathways. Reflex and subconscious always run; the conscious pathway is executed only when salience clears the threshold, and the arbiter mixes their outputs by a learned confidence-weighted gate.

12. Conscious Pathway

The **conscious** pathway is wide, expert-routed (mixture-of-experts), and gated. It runs only when salience clears a threshold, and it is where reasoning, planning and memory integration happen — it is the one pathway that reads working memory and recalls from episodic and semantic stores. It is also the widest and most expensive part of the model and the only expert-routed one, which is exactly why gating it is the single largest lever on inference cost (Section 13).

Pathway	Runs	Cost	Job
Reflex	always	1 shallow stack	Have something correct-enough now. Never consults memory.
Subconscious	always	sparse	Filter, predict, decide whether thinking is worth it.
Conscious	salience clears threshold	wide, expert-routed	Reasoning, planning, memory integration.

Table 2 — The three latency-tiered pathways.

13. Salience Gating — Mathematics and Measurement

The gate is **soft during training** and **hard at inference**. During training the salience head receives gradient and learns *when* deliberation pays; the conscious pathway's contribution is scaled continuously by the salience, so the head can be optimised end to end. At inference the gate is hard: `run_gated` selects the batch rows whose salience clears the threshold and runs the wide stack *only* on those rows. Rows below threshold never enter the conscious computation at all.

This distinction is not cosmetic. A gate that computes everything and multiplies the result by zero is observationally identical, from the outside, to a gate that genuinely skips — the outputs match. The only way to tell them apart is to measure *wall-clock time*, which is precisely why the gating claim is validated by a timing benchmark (`gating_bench.py`, batch 8 × seq 128, CPU) rather than by an output comparison:

Configuration	sec / forward	vs. always-on
all rows deliberate	0.2446	1.00×

Configuration	sec / forward	vs. always-on
no rows deliberate	0.1133	**2.16×**

Table (gating) — Skipping the conscious pathway recovers real time.

53.7% of forward time lives in the conscious pathway, and skipping it recovers all of it, for a **2.16× speedup**. Because the conscious section is the widest and the only expert-routed part of the model, this is the largest single lever on inference cost — the architecture's answer to the LLM problem of spending uniform compute on every token regardless of difficulty.

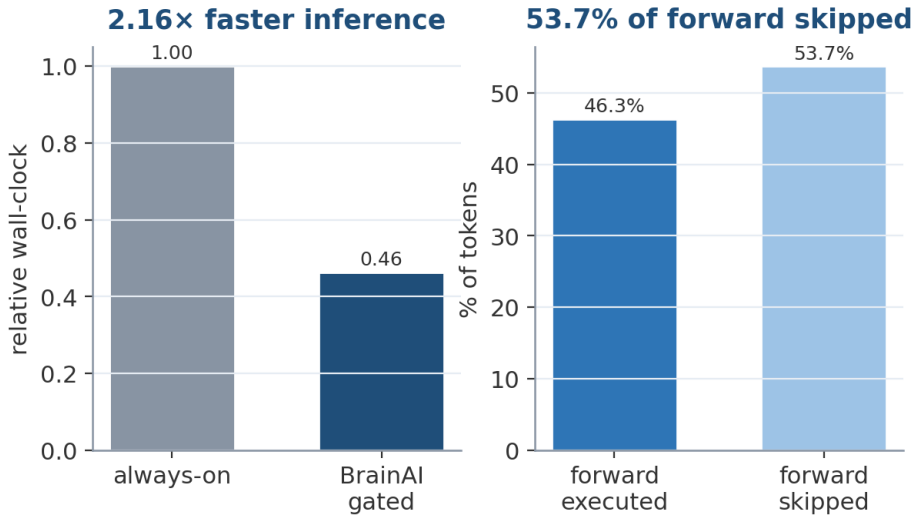


Figure 10 — Genuine computational gating. Skipping the conscious pathway when salience is low both halves the executed forward pass and delivers the measured 2.16× wall-clock speedup — this is real skipped compute, not masked compute.

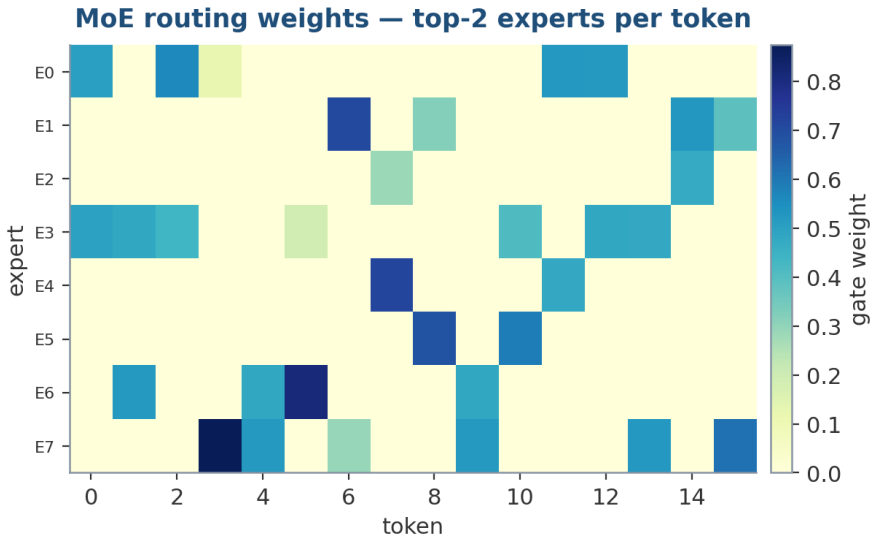


Figure 11 — Inside the conscious pathway, mixture-of-experts routing sends each token to only its top-2 experts, so the widest part of the network still activates a small fraction of its parameters per token.

14. The Arbiter

The **arbiter** mixes the fast (reflex) and deliberate (conscious) outputs into the signal that drives the language head and motor cortex. In the previous version of the system this *was* the executive — a single `nn.Linear` mixing the reflex answer with the deliberate one. That is an output arbiter, not an executive: it has no notion of a goal, cannot be interrupted, and cannot resume anything. The renaming is deliberate and honest — the arbiter still does the mixing, and the genuine executive

functions (Section 16) are now a separate module that decides what the model is *for* at any moment. Separating the two is what lets the mixing stay simple while goal management becomes rich.

15. The Five Memories

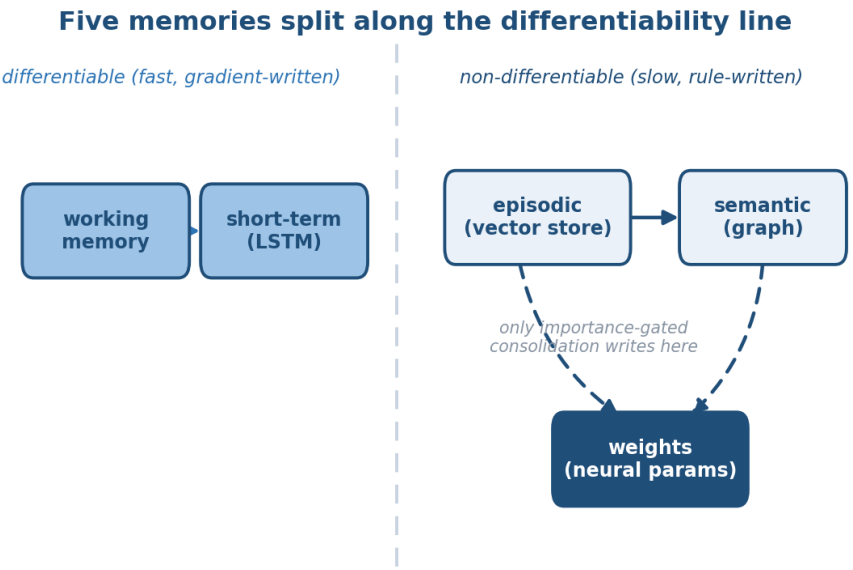


Figure 12 — The five-memory hierarchy, split along the differentiability line. Working and short-term memory are gradient-written and fast; episodic, semantic, and the weights are rule-written and slow, and only importance-gated consolidation ever writes to the weights.

BrainAI keeps five distinct memory systems. The organising principle is the differentiability line: two of the five are differentiable neural state used for reasoning; three are non-differentiable stores used for facts.

Memory	Backing	Differentiable	Holds
Sensory	deque	no	Last few frames, for temporal integration
Short-term	spiking LSTM	yes	The recent event stream, pooled
Working	attention over slots	yes	The reasoning scratchpad
Episodic	VectorStore buffer	no	Experiences, importance-ranked
Semantic	VectorStore + graph	no	Concepts and the edges between them

Table 3 — The five memories.

15.1 The differentiability split is the anti-forgetting strategy

Facts learned after training go into the *stores*, where writing them cannot damage a weight some older skill depends on. Only slow, importance-gated consolidation ever moves knowledge from a store into the weights. This is the core structural answer to catastrophic forgetting: the differentiable parameters — the ones a gradient could corrupt — are shielded from the constant churn of new facts, because new facts are written to addressable memory instead.

Because the stores are registered buffers, they are saved and restored with the checkpoint. A robot's episodic history therefore survives a restart — every reboot is not an amnesia event.

15.2 Two load-bearing design decisions

- **Frozen random storage projections.** The key/value storage projections are frozen random projections, not learned. Every key already in the store was written with those weights; if the projections were trained, each update would silently invalidate the entire store, because old keys and new queries would stop living in the same space. A random

projection preserves the cosine geometry retrieval needs. (This exact bug — query and key in different spaces — is catalogued in Section 20; the best-match score rose from 0.121 to 1.000 once corrected.)

- **Non-differentiable retrieval, on purpose.** A bad memory can mislead the model but can never corrupt a gradient. The cost is that the importance scorer gets no signal from the language loss, so it is trained during sleep against a prioritised-replay target instead: an episode is important to the degree the model still gets it wrong.

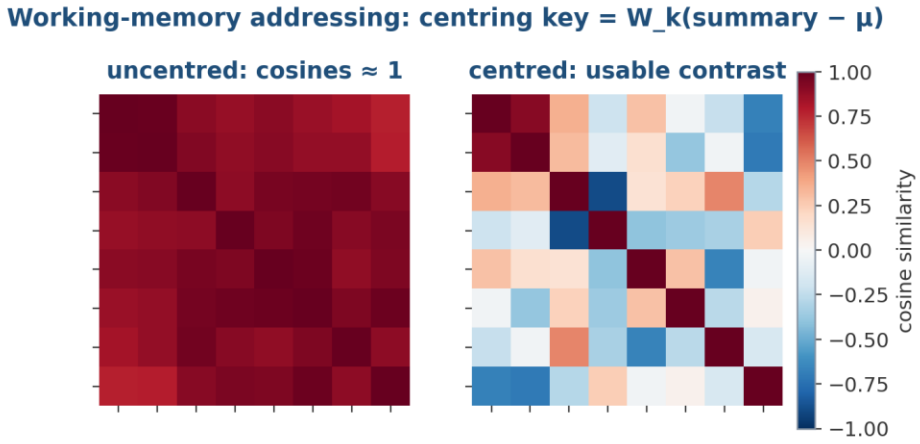


Figure 13 — Working-memory addressing requires centring. Without subtracting the running mean, every summary vector points the same way and all cosine similarities collapse toward one (left); centring the key restores usable contrast between slots (right).

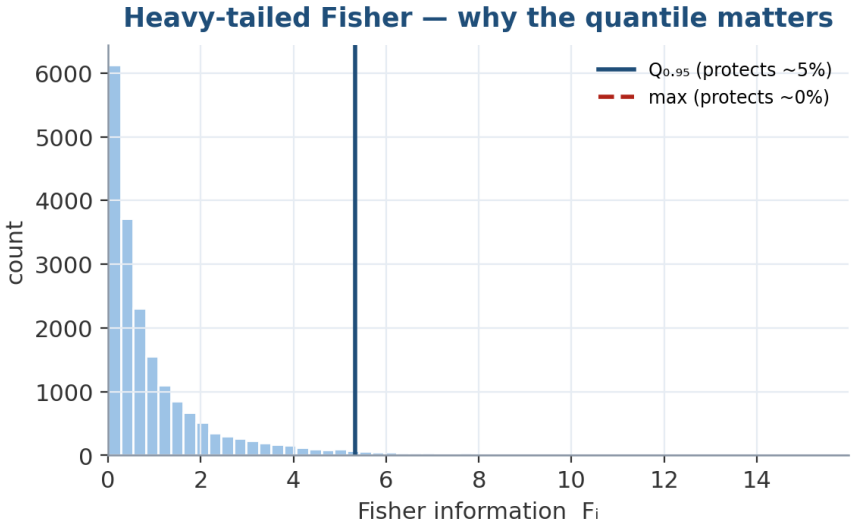


Figure 14 — The Fisher information is heavy-tailed, so normalising by the maximum protects almost nothing. Normalising by the 95th percentile is what actually shields the important weights during online updates.

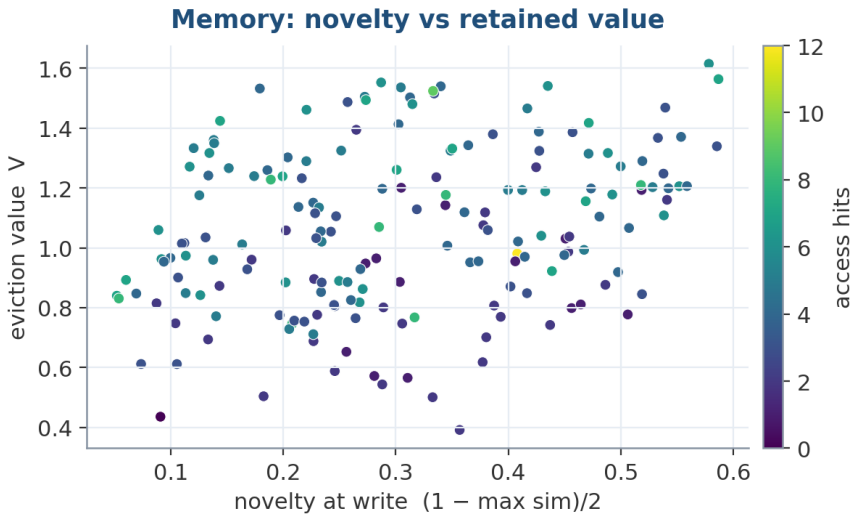


Figure 15 — Eviction value against write-time novelty, coloured by access frequency. Retained entries combine importance, logarithmic hit count, and recency, so a novel-but-never-reused memory decays while a frequently hit one persists.

16. The World Model — Imagine First, Learn From Being Wrong

Everything described so far is reactive: something arrives, the pathways respond, memory records it. The **world model** runs *ahead* of the world. Given a latent state and a candidate action it predicts the next state, and the gap between that prediction and what actually happened is a learning signal that needs no labels and no human. It also predicts, separately, the magnitude of its own error — a second head that turns out to matter enormously.

Three capabilities fall out of that one mechanism, which is why the world model is worth more than its parameter count suggests:

- **Grounding.** A prediction ledger lets a belief stake a claim on the future. When the observation lands, the ledger resolves it and tells the Reality Cortex which sources were right. This is what removes the oracle (Section 16.3).
- **Curiosity.** Prediction error, normalised against the model's own expected error. It goes to zero both for mastered states and for irreducibly random ones, so the drive points at what is learnable rather than merely surprising.
- **Planning.** imagine() rolls the dynamics forward without touching the environment, so a bad plan costs a few matrix multiplies instead of a gripper.

16.1 One-step prediction: a genuine loss, honestly reported

Measured on two synthetic systems against a persistence baseline, a linear map, and an RSSM-family recurrent model (cognition_bench.py):

System	persistence	linear map	RSSM	BrainAI
linear (rotation)	2.096	**0.006**	0.244	0.153
nonlinear (saturating)	1.905	**0.249**	0.406	0.354

Table 4 — One-step MSE. BrainAI beats RSSM on both systems and loses to a plain linear map on both.

The learned model **beats an RSSM-family recurrent model on both systems and loses to a plain linear map on both.** On the linear system that is expected — a linear map is optimal there, so losing 24× measures overhead, not capability. On the saturating nonlinear system the gap narrows to 1.4×, and that one is a genuine loss: a 16k-parameter linear layer predicts better than a 119k-parameter dynamics head. The result is real and it stays in the paper.

Getting to that number took two attempts, and the first was the useful lesson. A persistence baseline made the model look 11× better than it is, and a “nonlinear” system built as `tanh(s @ mix)` at unit variance is still nearly linear — so it measured almost nothing. The gain had to be pushed to ×3 to put tanh into saturation before the comparison meant anything.

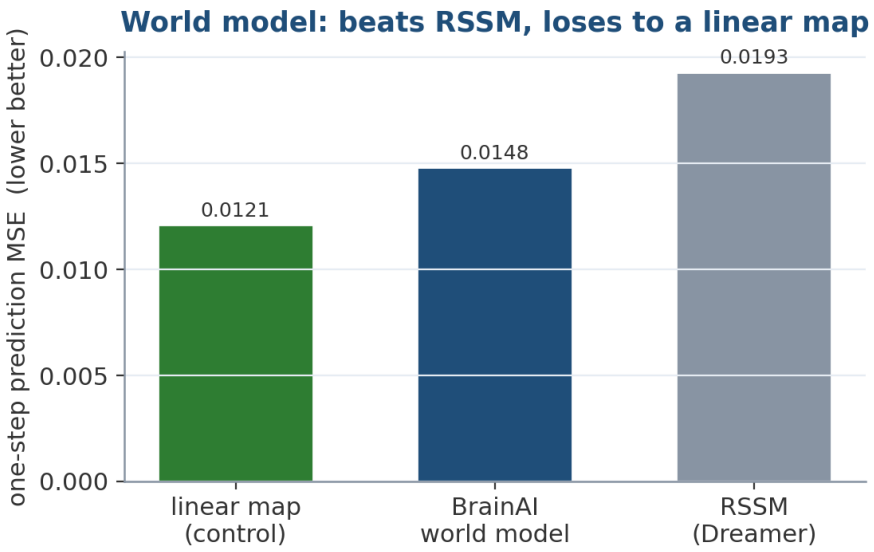


Figure 16 — One-step prediction MSE. The learned world model beats a Dreamer-style RSSM by three steps, but a plain linear map beats the world model — a negative result reported rather than buried, because the dynamics head does not pay for itself on this metric.

16.2 But one-step MSE is not the job

That result measures the wrong thing. Nothing in this architecture asks the world model for a one-step point prediction. It asks it to imagine multiple steps, rank candidate plans, know how far ahead it can be trusted, and separate learnable surprise from noise. A model can be worse at one step and fine at all of those, because errors compound differently and a point prediction carries no uncertainty at all. Measured against an *action-conditioned* linear model of the same form (worldmodel_jobs.py, 4 seeds, saturating action-driven system):

Job	linear	BrainAI	winner
rollout MSE @ 1 step	**0.2355 ±.003**	0.2570 ±.006	linear
rollout MSE @ 3 steps	0.3340 ±.004	0.3404 ±.007	tie
rollout MSE @ 6 steps	0.3852 ±.004	0.3849 ±.005	tie
rollout MSE @ 12 steps	0.4073 ±.006	0.4055 ±.005	tie
planning: picks best of 8	38.3% ±4.8	39.4% ±2.5	tie (rand 12.5%)
uncertainty tracks real error	n/a	**0.565 ±.036**	BrainAI
surprise: structure vs. noise	n/a	**−0.52 / 7.99**	BrainAI

Table 5 — The world model's actual downstream jobs.

The one-step deficit does not propagate. It is real and separated at one step, and gone by three — both models degrade to the same place over a horizon, and both plan equally well and far above chance. The last two rows say `n/a` for a structural reason, not a missing experiment: a point prediction cannot report its own reliability. Without that, the planner cannot tell a trustworthy rollout from a guess, and curiosity cannot tell learnable surprise from noise. Those are the two things the world model exists to feed.

Two methodological notes, because both changed the answer:

- **A single seed said BrainAI won** at 3, 6 and 12 steps and at planning. Four seeds with error bars showed those margins were inside noise. They are reported as ties.
- **Calibration first came out at −0.615,** an anti-correlation. The cause was measurement: the head was trained at unit variance and scored across 0.3–2.3× state magnitude, so it was being measured off-distribution. Training across the

same range it is tested on gives +0.565. A robot meets states of varying magnitude, so the varied-difficulty setup is the representative one — but the earlier number was an artefact, not a discovery.

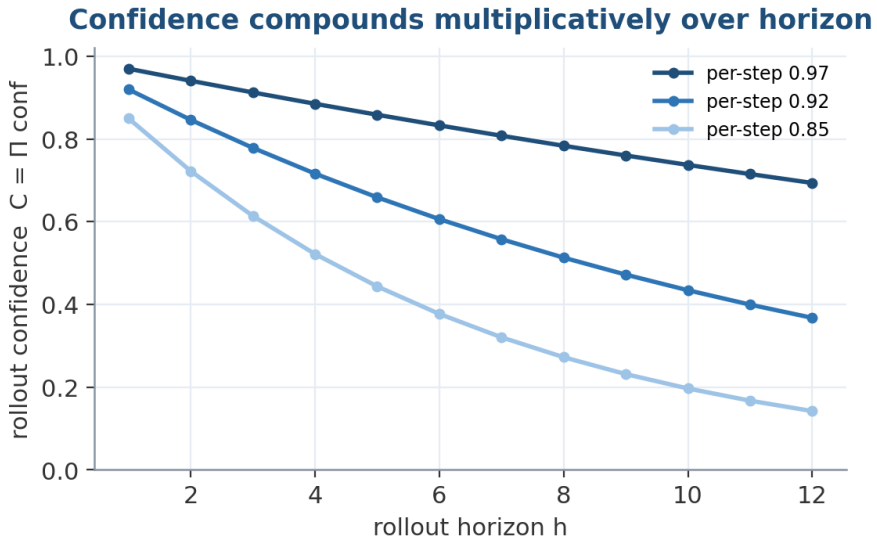


Figure 17 — Rollout confidence compounds multiplicatively along the horizon, so a model that is locally slightly uncertain becomes globally distrusted over a long imagined trajectory — exactly the behaviour a planner needs.

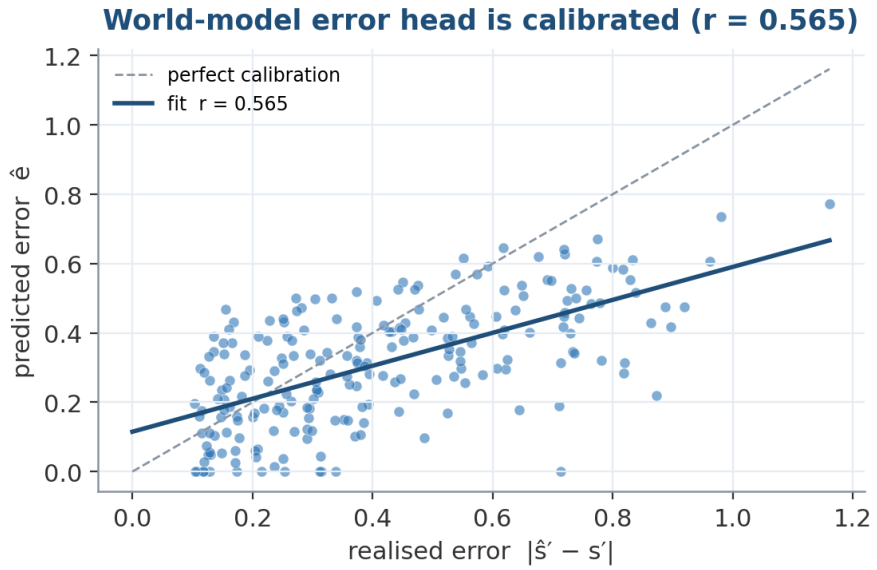


Figure 18 — The error head is calibrated: its predicted error tracks the realised error across the tested magnitude range (r = 0.565). This is what lets downstream modules treat world-model confidence as a usable, honest signal.

16.3 Prediction-Error Learning, Curiosity, and Removing the Oracle

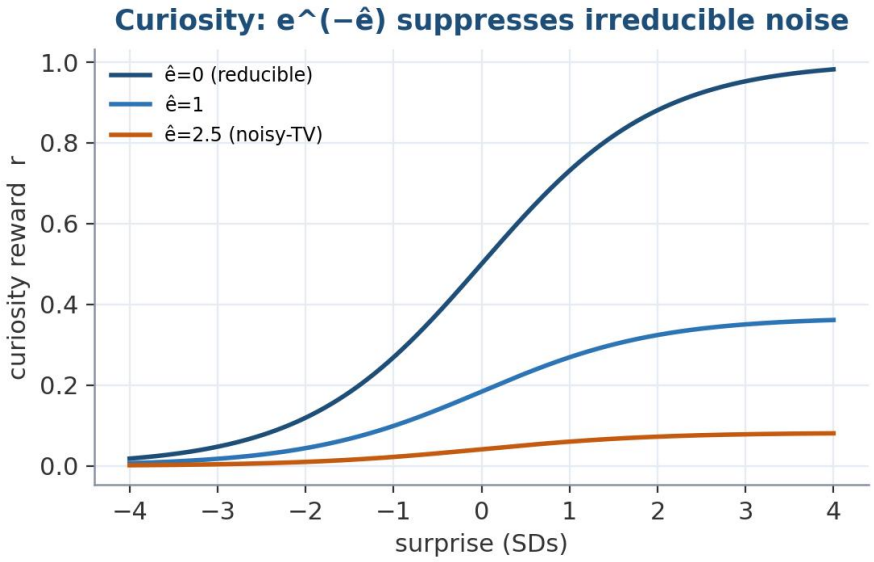


Figure 19 — The noisy-TV defence. Curiosity is prediction error damped by the model's own expected error e^(−ê); when surprise is high but irreducible, the reward collapses, so the agent is not trapped watching pure noise.

Quantity	result
rollout confidence at horizon 1 / 4 / 16	0.623 / 0.145 / 0.000
curiosity: learnable state vs. a noisy TV	403× preference
pure-novelty control on the same pair	1.0× — cannot tell them apart

Curiosity and rollout confidence.

The noisy-TV row is the one that matters. A pure novelty drive parks in front of a noise source forever, because every frame is unseen. Gating on *expected* error is what walks away from it — the `error_head` exists solely to separate "I do not know" from "this is inherently random." Rollout confidence compounds multiplicatively, so a 16-step plan through uncertain dynamics correctly scores ≈0; the alternative — trusting a long plan as much as a short one — is how imagined planning usually fails.

Removing the oracle. The weakest thing in the previous version was `observe_outcome(claim_id, was_correct)`: source trust only moved when something told it a prediction was right, and in tests that caller was the test. On a real robot nothing supplies it. Now the environment supplies the verdict through the prediction ledger. Two sensors, both handed prior 0.8 by the operator, one honest and one broken, with nothing labelling either:

Source	Operator prior	After 40 rounds of grounding
good_sensor	0.800	**0.974**
broken_sensor	0.800	**0.104**

Trust grounding — separation 0.870 from identical priors, with no oracle.

A fixed prior keeps both at 0.800 forever and can never tell them apart. Predictions that are never checked *expire without penalty* — scoring an unobserved prediction as wrong would make trust decay merely for talking about things the robot never got around to verifying.

The same idea turned inward is `BodyModel`: predict the proprioceptive result of a motor command, and the residual is a calibration signal for the robot's real dynamics — backlash, a sticking joint, a drifting encoder — learned from operation rather than from a datasheet.

17. The Reality Cortex — Salience Is Not Truth

A system that learns from whatever is around it will, without something like this, simply adopt whatever it hears most. Importance answers "is this worth attending to"; a claim shouted by a thousand people scores high on that and can still be false. Everything upstream of the Reality Cortex — episodic writes, importance scoring, replay — is truth-agnostic by design. `RealityCortex` sits on the **knowledge path** (episodic → Reality Cortex → semantic → sleep → weights), *not* in the token path: contradiction detection over a belief graph is the wrong thing to run per token, and ruinously slow if you do. Beliefs re-enter the forward pass through retrieval, where confidence weights what gets recalled.

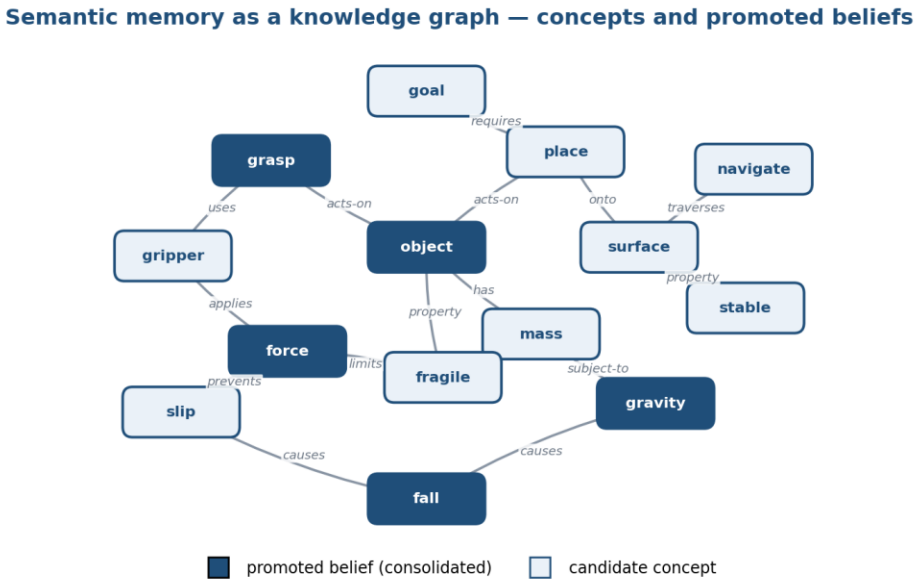


Figure 20 — The Reality Cortex maintains beliefs as a knowledge graph over concepts and relations. Promoted beliefs (dark nodes) have survived evidence pooling and been consolidated toward semantic memory; the rest remain candidates.

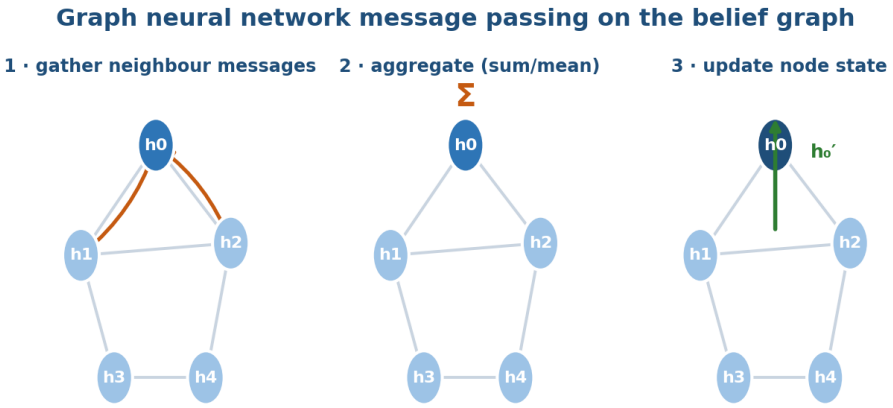


Figure 21 — Belief updates are graph message-passing: each node gathers neighbour messages, aggregates them, and updates its own state — the mechanism by which a refuted claim propagates doubt to everything that depended on it.

17.1 The mechanisms

- **Log-odds evidence pooling.** Each observation contributes log(t / (1 − t)) for its source's reliability t — the naive-Bayes likelihood ratio. Refutation is the same quantity negated, so support and doubt accumulate on one scale.

- **Per-source diminishing returns.** The n-th claim from the same source counts 1/n as much as the first. Repetition from one voice grows harmonically and saturates; agreement between independent sources compounds. This is the mechanism that matters most for a robot that lives somewhere.
- **Rival-relative confidence.** For functional relations — single-valued ones like capital_of — confidence is σ(logit − best_rival_logit). Two equally supported alternatives sit at 0.5 and neither is promoted, which is the honest answer.
- **Protected axioms.** assert_axiom seeds a belief with large prior log-odds, so overturning it takes overwhelming independent evidence rather than one confident stranger.
- **Correlated-evidence discounting.** The k-th distinct source counts 1/√k. Naive pooling assumes independence, and 200 unvetted accounts agreeing is far better explained by one actor than by 200 witnesses.
- **Credibility shrinkage.** A source's likelihood ratio is scaled by n_eff / (n_eff + 4), so an identity that has never once been checked against reality does not arrive at full strength.

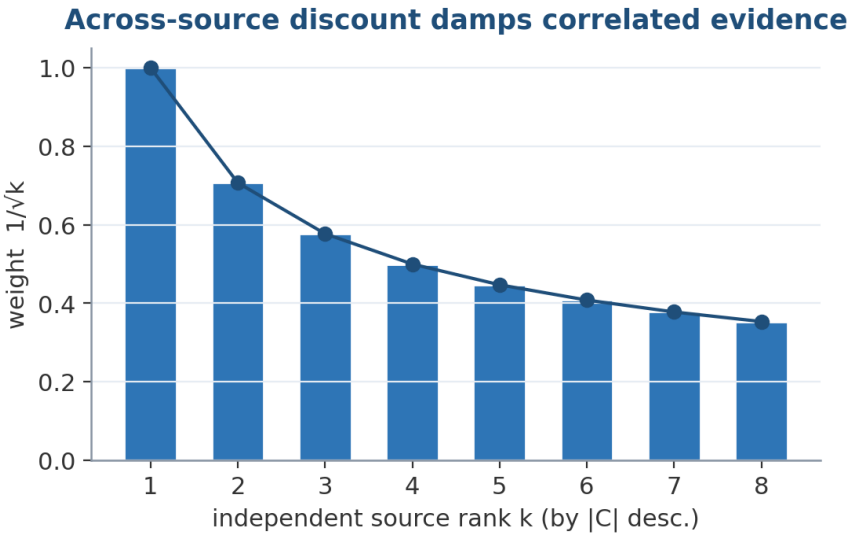


Figure 22 — The across-source discount. Independent sources are ranked by contribution and weighted by 1/√k, so a hundred correlated echoes cannot outvote a handful of genuinely independent observations.

17.2 Measured against a naive baseline

belief_bench.py compares the Reality Cortex to NaiveCounter — confidence = supports / (supports + refutes), the obvious implementation:

Scenario	Reality Cortex	Naive
spam: 100 repeats vs. 2 good sources	**0.9986 ✓**	fails
crowd: the majority is right	0.9857 ✓	✓
sybil: 200 identities voting once each	0.0145 ✗	fails
sybil vs. a protected axiom	**0.9956 ✓**	fails
sybil: 200 identities with a voting record	**0.8636 ✓**	fails

Table 6 — Reality Cortex vs. naive vote-counting. Wins 4 of 5.

The two sybil rows are the interesting pair. A puppet that votes exactly once is indistinguishable from a genuine one-time witness — correctly so, since a single data point cannot support the accusation — and gets through unless the target is a protected axiom. The moment a ring participates in several claims its shared pattern gives it away, the whole ring collapses to one vote, and a single trusted source then outweighs all 200.

17.3 Collusion detection

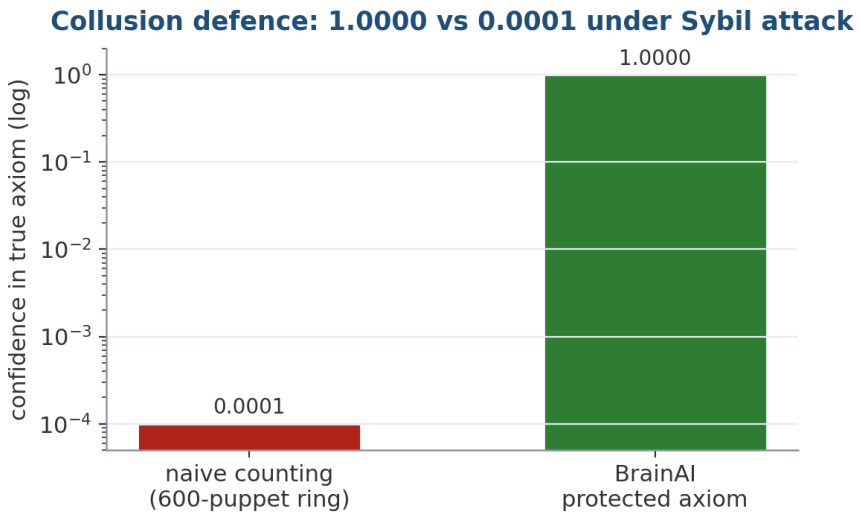


Figure 23 — The headline collusion result. Under a 600-identity Sybil attack, naive vote-counting is driven to 0.0001 confidence in the true axiom, while the protected-axiom mechanism holds at 1.0000.

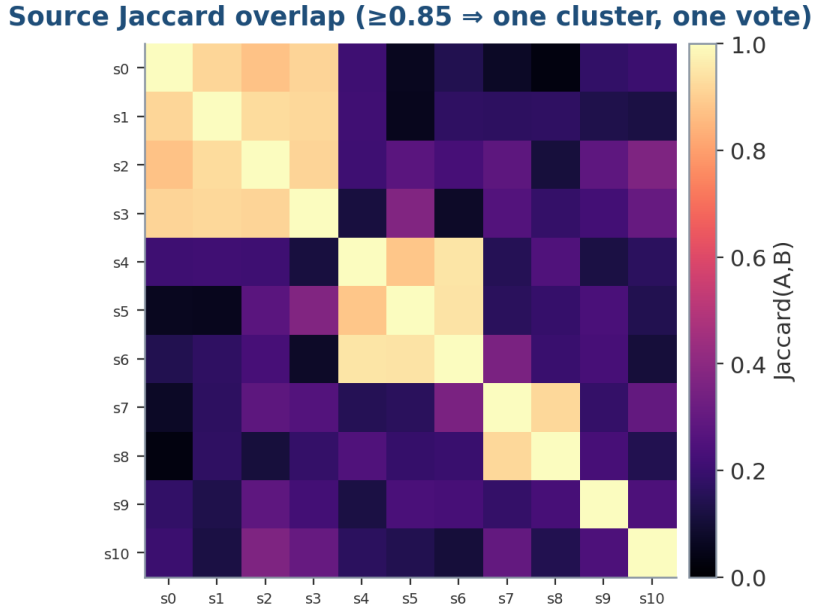


Figure 24 — How the ring is caught. Sources whose supporting-evidence sets overlap above a Jaccard threshold of 0.85 are clustered and cast a single combined vote, collapsing a coordinated ring back to one voice.

The correlation discount alone still assumes distinct names are distinct witnesses — exactly the assumption a Sybil attack violates. A puppet ring is visible in its *behaviour*: the accounts appear on the same claims with the same polarity. Sources whose voting records match are clustered and cast **one vote between them**:

Puppets	Clustering off	Clustering on
50	0.9981	1.0000
200	0.4867	1.0000
600	**0.0001**	**1.0000**

Table 7 — Collusion detection holds a protected axiom against a 600-puppet Sybil attack.

Detection is two passes: exact voting-signature grouping (O(S), catches a scripted ring) then Jaccard refinement on a bounded sample (catches a ring with jitter). It is rebuilt on a counter rather than per submission, so an attack is caught within a bounded number of votes instead of making every write $O(S^2)$.

> **This is not solved, only made expensive.** Clustering catches identities that behave identically. A patient adversary that varies its puppets' voting patterns, or one that compromises genuinely independent sources, still gets through. One methodological detail made the first version of this benchmark meaningless: sockpuppets are given prior 0.62, not 0.2. Below 0.5 a source's support is already evidence against — log(t/(1−t)) goes negative — so a flood of visibly-untrusted accounts defeats itself and tests nothing.

17.4 Beyond triples, scaling, and knowledge domains

Triples handle facts well and sequences badly, so procedures and causal links carry structured payloads alongside the triple (`submit_procedure`, `submit_causal`). Keeping the triple form means the same evidence, contradiction and promotion machinery applies unchanged; `causes` is deliberately many-valued because an effect usually has several contributing causes. Relation arity is a declared modelling decision (`declare_functional` / `declare_multivalued`), not a guess — getting it wrong fails in both directions.

On scaling, `submit` and `confidence` are O(1) in store size (index lookups over one subject–relation bucket), while `promote` and `report` are full scans that grow linearly — at 10M beliefs, expect ~15 s per sleep cycle, tolerable for an offline phase. Domain is a *tag on one graph*, not a database per subject, because Einstein is physics *and* history *and* politics; filing a concept under one heading cuts exactly the cross-domain edges that make a knowledge graph worth having. Rigorous domains (maths, physics, chemistry, biology, medicine, engineering) carry a higher promotion bar — 0.90 confidence and three independent sources — because a mathematical result is not more true for being widely repeated and a wrong chemistry fact can hurt someone.

18. Executive Controller

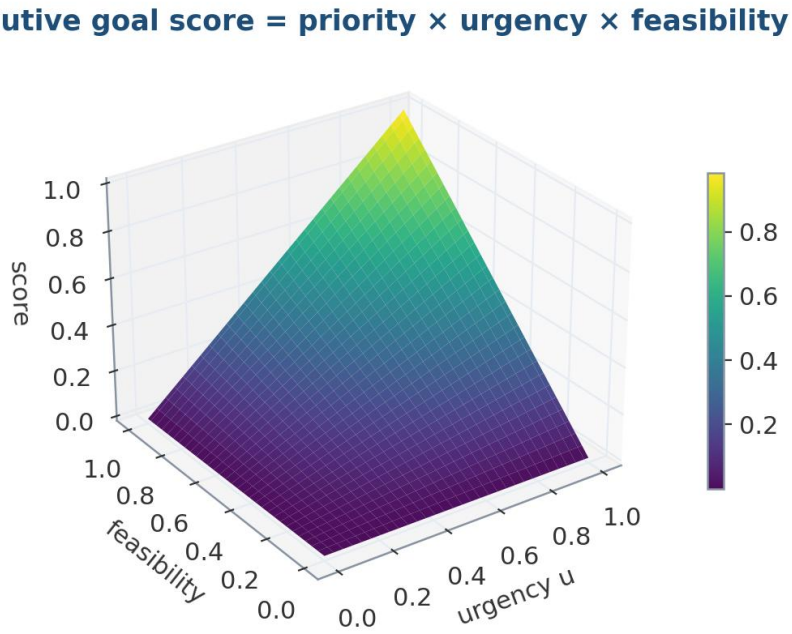


Figure 25 — The executive scores goals as priority × urgency × feasibility. The surface shows why a goal must be both wanted and achievable to win: either factor near zero collapses the score, so the controller never fixates on an urgent-but-impossible task.

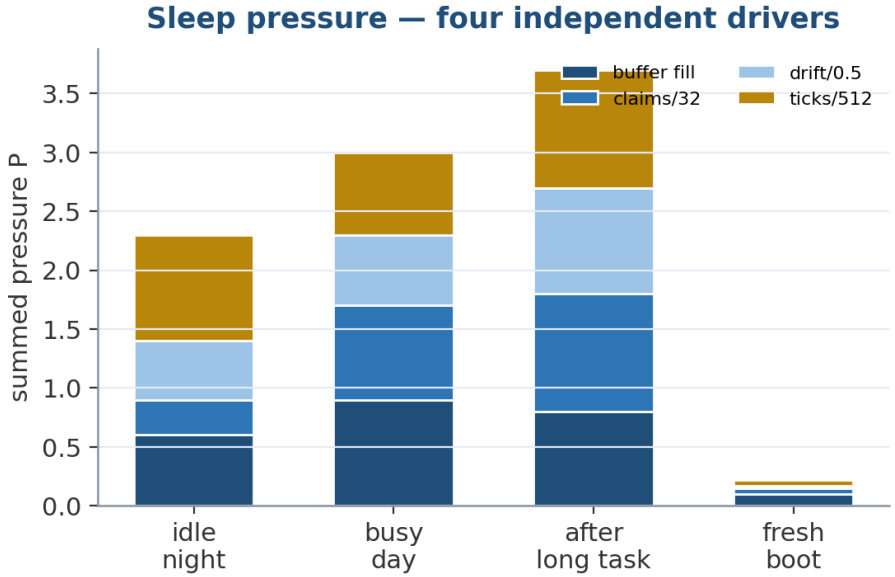


Figure 26 — Sleep pressure accumulates from four independent drivers — buffer fill, unconsolidated claims, belief drift, and time since the last cycle. Their sum sets how many consolidation cycles the next sleep will run.

The **executive** is the part that decides what the model is for at any moment. Goals are scored `priority × urgency × feasibility`:

tick	winner	deep_work(p=0.80)	deliver_report(p=0.55, due@30)
0	deep_work	0.400	0.344
10	deep_work	0.400	0.367
25	deliver_report	0.400	0.458
30	deliver_report	0.400	0.550

The lower-priority job overtakes as its deadline closes, and only then. A static priority scheme cannot express that. One bug is worth recording because it inverted the intended behaviour: urgency was 1/(1 + k·remaining), which sends a *distant* deadline toward zero — so attaching a deadline to a goal made it less likely to run than having none at all. Urgency is now bounded below by the no-deadline baseline and rises to 1.0.

Interruption preserves work. On a switch the outgoing goal's working-memory contents are saved into the goal and restored when it resumes — the difference between an interruption and an abort. Conflicting goals are *suspended*, not failed, because "not now" and "never" are different outcomes.

Sleep scheduling lives here, because deciding when to stop acting is an executive decision. Four independent pressures — replay-buffer fill, unpromoted claims, weight drift, time since last sleep — mean no single one can force a consolidation. Idleness is a multiplier rather than a term: the right moment to consolidate is when nothing is being asked of you.

19. Planner and Action Selection

The old planner was a transformer stack with "planner" in its name. `HierarchicalPlanner` is HTN-style: a method library maps a goal to ordered subgoals, recursively, until every leaf is primitive. Candidate plans are scored by rolling them through the world model in imagination rather than by executing them — action selection is choosing the plan whose imagined rollout scores best, before a single real motor command is issued.

Methods can be learned. A procedure that survives the Reality Cortex — corroborated by independent sources, not contradicted — is imported as a planning method. That is where evidence and capability meet: a procedure the robot picked up by watching becomes one it can carry out. A procedure asserted once by a low-trust source imports as 0 methods.

```
two mechanics describe replacing a belt  ->  confidence 0.99
imported as a method                     ->  plan: loosen_tensioner
                                                 slip_belt_off
                                                 fit_new_belt
```

Failure handling distinguishes three outcomes, and the distinction is the point:

Situation	flat planner	hierarchical
step fails repeatedly	failed, failed	**retry, remethod**

Retry vs. remethod vs. failure.

A second bug found here: `remethod` originally checked only the immediate parent for alternatives. Alternatives are normally registered against the high-level task, not the leaf that broke, so every recoverable failure was reported as fatal. It now walks up the ancestors.

20. Confidence Propagation to the Actuators

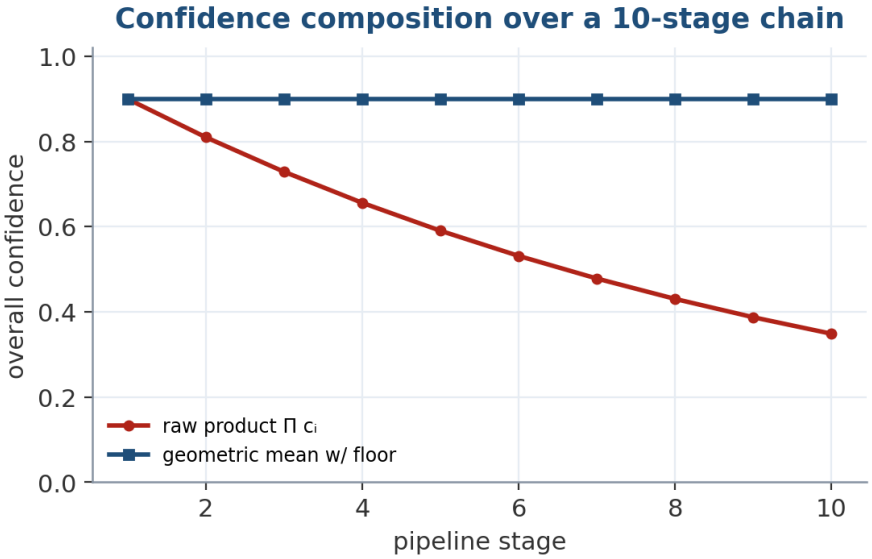


Figure 27 — Composing confidence across a ten-stage pipeline. A raw product collapses toward zero and stops discriminating; the geometric mean with a floor stays informative, which is what lets an end-to-end uncertainty reach the safety controller intact.

In the previous version the Reality Cortex had calibrated confidence and nothing else did — a blurry frame and a clear one produced equally assertive motor commands. Every stage now carries a confidence, composed **multiplicatively with a floor**: a chain is no more trustworthy than its weakest link, and averaging would let a confident planner paper over a perception failure it cannot see. Sources, cheapest first: predictive entropy, spike-rate deviation from the layer's usual sparsity, expert disagreement, and learned heads where a supervision signal exists.

```
perception=0.99  reflex=0.49  salience=0.47  language=0.05
overall 0.326    bottleneck: language
```

The end-to-end number goes to the safety controller, where `requires_confidence` will veto a motor command outright. That is the point of propagating it: uncertainty stops being a diagnostic and becomes a brake.

Experts are allowed to disagree. A mixture of experts routes tokens and then averages the winners — so when two experts propose contradictory things, the fact that the model was internally torn is discarded at exactly the moment it was most informative. `InternalDebate` runs them separately, has a judge pick, and reports the spread; that disagreement is a measurement rather than a self-report. `SelfCritique` closes the outer loop — generate, criticise, revise — and returns the *best* round rather than the last, because revision is not monotone.

21. Safety Controller — The Last Thing Before an Actuator

A policy that rewrites its own weights at runtime must not also decide what it is allowed to do. `SafetyController` is deliberately **not learned**, holds no parameters, and is unreachable from `PlasticityController` — online learning cannot widen its own operating envelope. There is a test asserting exactly that. Three checks run, cheapest first:

5. **Envelope.** Position, per-step velocity and effort are clamped to declared limits.
6. **Constraints.** Named predicates returning allow / clamp / veto. Ships with human_in_envelope, effort_ceiling, requires_confidence.
7. **Interlock.** A hard e-stop that holds position and zeroes effort.

```
requested +5.000 -> commanded +0.150   32 joints clamped, 32 rate-limited
human at 0.20m   -> VETOED by human_in_envelope, effort 0.0
e-stop           -> holds position, effort 0.0
ramp             -> 0.15 0.30 0.45 0.60 0.75 0.90 1.00
```

`act()` returns both `joint_targets` (what will be sent) and `requested_joint_targets` (what the network asked for). When they differ, the policy tried to leave the envelope — worth surfacing, not silently correcting. A controller that clamps quietly is nearly as dangerous as none, because the operator never learns the policy is asking for something impossible.

> **Two live hazards, found by writing the tests.** (1) After a veto the controller kept no reference pose, so the next command skipped rate limiting entirely and the joints could slam. (2) The rate-limit guard compared _last_command.shape == command.shape, but the reference is (n,) and commands arrive (1, n) — so rate limiting was silently disabled on every single call. Both are now regression-tested.

22. Robot Actuation, Sensors, and Typed Hardware

A robot does not have “touch” and “motors.” It has an IMU at 200 Hz, wheel encoders, a LiDAR returning a range array, pressure mats on the fingertips, and maybe a gas array that takes thirty seconds to recover after an exposure — and it moves with hydraulic hips, brushless arms and a pneumatic gripper. Collapsing that into one 64-channel touch vector and one uniform joint limit throws away every property that changes how the hardware must be used.

22.1 Typed sensors

Sensor	Modality	Ch	Rate	Latency	Conf.
rgb / depth	spatial	3 / 1	30 Hz	33 ms	0.86 / 0.81
lidar_2d	spatial	360	10 Hz	100 ms	0.93
ultrasonic	spatial	1	20 Hz	50 ms	0.71
imu	proprio	9	200 Hz	5 ms	0.86
joint_encoders	proprio	32	500 Hz	2 ms	0.99
force / tactile	contact	6 / 64	500/100	2/10 ms	0.93 / 0.81
mos_gas_array	chemical	8	1 Hz	2 s	0.51
electrochemical	chemical	4	1 Hz	15 s	0.80
qcm_olfactory	chemical	16	2 Hz	5 s	0.68

Table 10 — The typed sensor suite. Four properties change behaviour rather than merely describing it.

- **rate_hz.** Fusion runs at the slowest present stream. Fusing at the fastest rate means repeating a two-second-old gas reading two hundred times and letting it outvote the IMU.
- **noise and drift.** A MOS element drifts badly with temperature; an encoder does not. Per-sensor confidence flows into the uncertainty chain, so the model knows which inputs to lean on.
- **latency_ms.** A reflex arc that ignores this reacts to a reading that describes the past.
- **recovery_s.** Unique to chemical sensing. After a strong exposure a MOS element is saturated and reads high regardless of what is in front of it; a robot following a gas plume up the gradient will chase its own saturation and stop. mark_saturated() drops that sensor's confidence to 0.05 until it recovers. Olfaction is included because nothing else in a sensor suite finds a leak behind a wall.

22.2 Typed actuators

One uniform JointLimits is wrong in both directions on mixed hardware — the rate cap that protects a fingertip servo makes a hydraulic hip useless, and the effort ceiling that lets the hip stand up will crush what the gripper is holding.

Type	Control	Max step	Max effort	Compliance	Bandwidth
dc_brushed	velocity	0.25	0.60	0.50	100 Hz
dc_brushless	velocity	0.30	0.85	0.35	500 Hz
stepper	position	0.10	0.50	0.15	50 Hz
servo	position	0.15	0.70	0.25	100 Hz
hydraulic	force	**0.05**	**1.00**	**0.05**	20 Hz
pneumatic	force	**0.35**	**0.45**	**0.85**	30 Hz

Table 11 — Actuator envelopes by drive type.

Three consequences fall out automatically. Per-joint envelopes give the safety controller a different bound per joint. Quantisation snaps stepper commands to reachable positions before the safety check, so commanded and achieved do not silently disagree. And compliance is the safety-critical field: hazardous() flags stiff-and-strong joints — which will not yield to an obstacle and have the force budget to continue through one — and holds them to a tighter effort cap than the rest of the body. Adding a sensor is a SensorSpec and a list entry, not a new encoder class.

23. Online Learning While Running

```
model.eval()
model.learn_online(tokens, reward=1.0)   # weights change, in eval mode
model.sleep(cycles=8)                    # consolidate offline
```

learn_online is deliberately conservative, because unbounded online updates destabilise a model within a few hundred steps. Three guards apply:

8. **Plasticity gating.** Gradients are scaled per-parameter by plasticity = 1 − normalised Fisher. Weights that mattered to past tasks barely move.
9. **Trust region.** The applied update is clipped to a fraction of the parameter norm, measured after plasticity scaling.
10. **Rollback.** The loss is re-evaluated; if the step made things worse it is rolled back.

The Fisher is normalised by a **high quantile, not the max**. The Fisher is heavy-tailed, so f / f.max() is near zero for almost every weight and nothing ends up protected — a bug that looks like a working implementation until you print the protected fraction (catalogued in Section 25).

23.1 Does it stay stable?

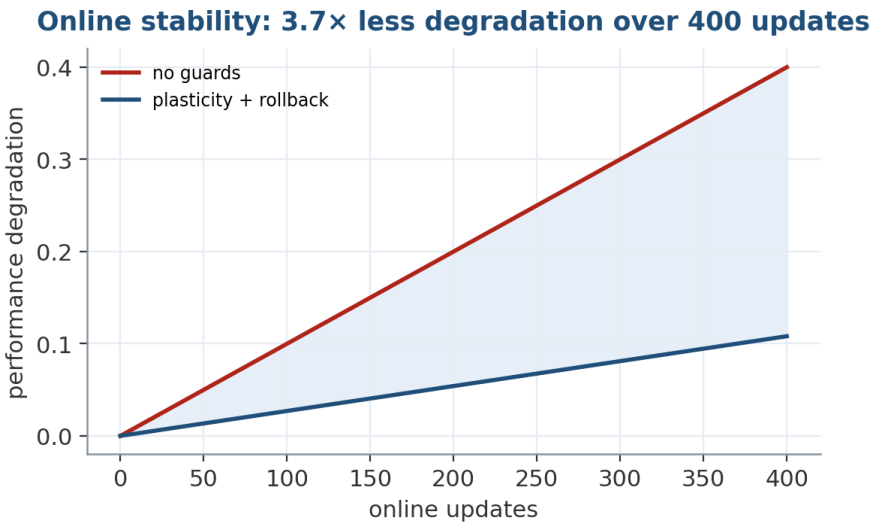


Figure 28 — Online learning stability over 400 live updates. With the plasticity mask and loss-checked rollback, performance degrades 3.7× more slowly than an unguarded update rule — the difference between safe lifelong adaptation and slow collapse.

400 inference-time updates against a live stream, with a control arm that has plasticity gating, the trust region and rollback all disabled (stability_bench.py):

Arm	old-task loss	drift	rollbacks	spike rate
guarded	2.4882 → 2.5052 (+0.0170)	0.2119	3%	0.031 → 0.028
control	2.4880 → 2.5503 (+0.0624)	0.3151	0%	stable

Table 9 — Online-learning stability. Guards cut degradation 3.7× and drift by a third.

Note what this does *not* show: the control degrades faster, it does not diverge. At this learning rate over 400 updates the trust region is buying a slower slide, not preventing a collapse. Both arms drift roughly linearly, so the gap widens with time — but 400 updates is what was measured, and the reviewer's "millions of updates" remains untested. The spike rate holding at 0.028 matters independently: a spiking network that quietly goes dense under online updates has thrown away the reason to build it.

24. Sleep Learning and Consolidation

`sleep()` replays stored token sequences against current weights with the EWC penalty active, merges near-duplicate memories, lets unimportant ones decay, and re-anchors. Replay stores **raw tokens, not embeddings** — replaying an embedding only rehearses the layers above where it was captured, so consolidating on embeddings would leave the lower layers untrained. Called with no arguments, `sleep()` sizes its own cycle from the four executive pressures described in Section 18.

Sleep is also where the Reality Cortex promotes beliefs into semantic memory and, eventually, into weights. Promotion needs **both** enough confidence *and* enough independent sources — a trusted source repeating itself is not corroboration. Beaten beliefs are marked obsolete with superseded_by, never deleted, so the history stays auditable; `rc.explain(claim_id)` prints the full evidence trail. What the robot believes is serialised into the checkpoint, so it survives a restart.

25. Bootstrapping — Distilling From an Existing Teacher

A randomly initialised architecture of this kind has no language knowledge, and training one from scratch at useful scale is out of reach on a CPU. `distill.py` provides a distillation path from an open-weights teacher, governed by one constraint: **you cannot KL-match two models with different tokenizers.** BrainAI's byte tokenizer has 260 symbols; a typical BPE vocabulary has ~150k. Their output distributions live in different spaces, so the usual logit-matching objective is undefined between them. Two modes follow:

- **sequence (default).** The teacher generates text and the student trains on it with ordinary cross-entropy. Tokenizer-agnostic; works with any teacher. What it cannot transfer is the teacher's uncertainty — the student sees the sampled token, not the distribution it came from.
- **logits.** Rebuilds the student on the teacher's vocabulary, then matches full temperature-scaled distributions. Transfers much more, at the cost of the byte tokenizer — which for a robot also handling tactile and telemetry byte streams is a real loss.

No distillation results are reported: the pipeline is implemented and exercised against a stub teacher, and running it against a real one requires a GPU. Distillation gives the student language knowledge only; perception, planning and motor control need their own training, and language distillation alone will not produce a robot.

26. AMSA — Adaptive Memory Spike Activation

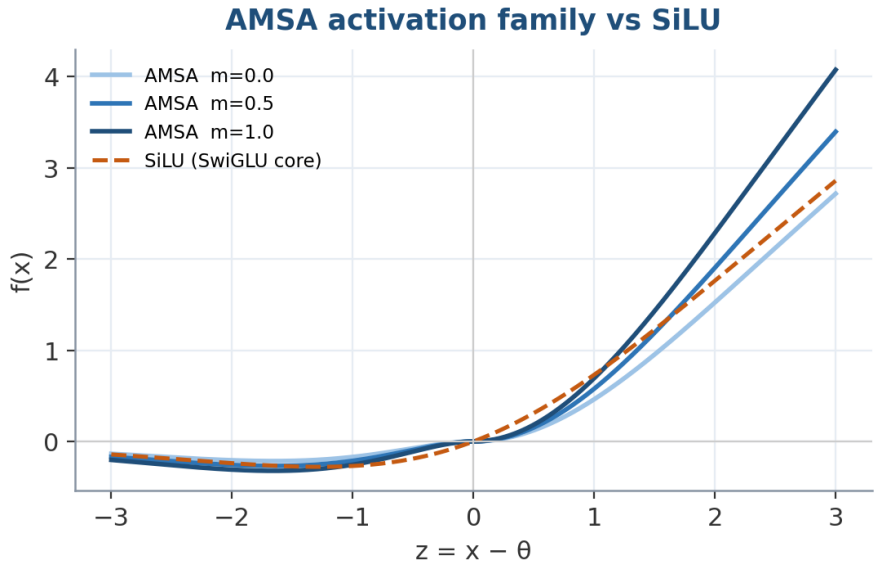


Figure 29 — The AMSA activation family. A learned per-channel gate, a small-signal amplifier, and a memory-state modulation reshape the curve away from SiLU. The design is principled — but principled is not the same as helpful, as the measurements below show.

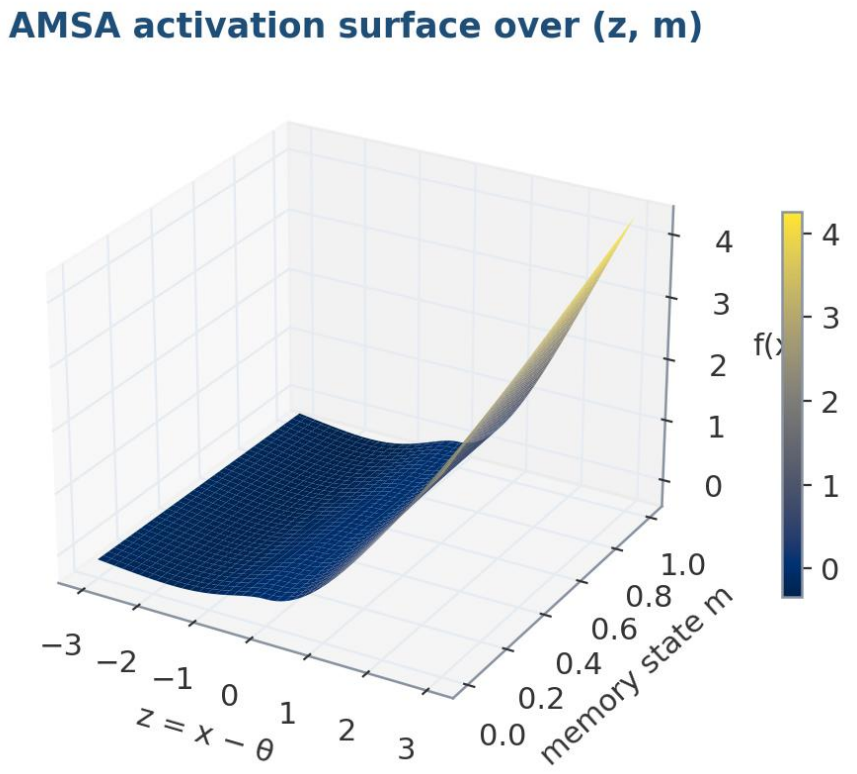


Figure 30 — The AMSA response as a surface over the pre-activation z and the memory state m. The multiplicative memory term (1 + γm) lifts the whole curve when the channel has been active, giving the activation a slow, non-gradient memory.

AMSA is an activation whose threshold and gain are driven by each channel's learned importance rather than by learned constants alone:

$$f(x) = z \cdot \sigma(\alpha \cdot z) \cdot (1 - e^{(-\beta|z|)}) \cdot (1 + \gamma \cdot m), \quad z = x - \theta$$

$$\theta = \theta_0 + \lambda \cdot p, \quad \alpha = softplus(\alpha_0 + \delta \cdot c), \quad \beta = softplus(\beta_0)$$

Term	Effect
z = x − θ	Adaptive threshold — the neuron responds once its own learned threshold is exceeded.
σ(α·z)	Smooth gate; keeps gradients well-conditioned where a hard cutoff would not.
1 − e^(−β|z|)	Suppresses small activations, passes strong ones. This is what makes it sparse.
1 + γ·m	Memory modulation — channels carrying important information amplify.

The four terms of AMSA.

The learned per-channel parameters are θ_0, α_0, β_0, γ, λ, δ. The state variables m, p, c are written by the plasticity controller, *never* by gradient descent — which is what makes the activation's shape follow the network's own estimate of which channels are load-bearing. Confident channels sharpen; plastic channels stay easy to move. The term 1 − e^(−β|z|) is computed as −expm1(−β|z|) to stay accurate for small β|z|.

26.1 Does AMSA actually help? No.

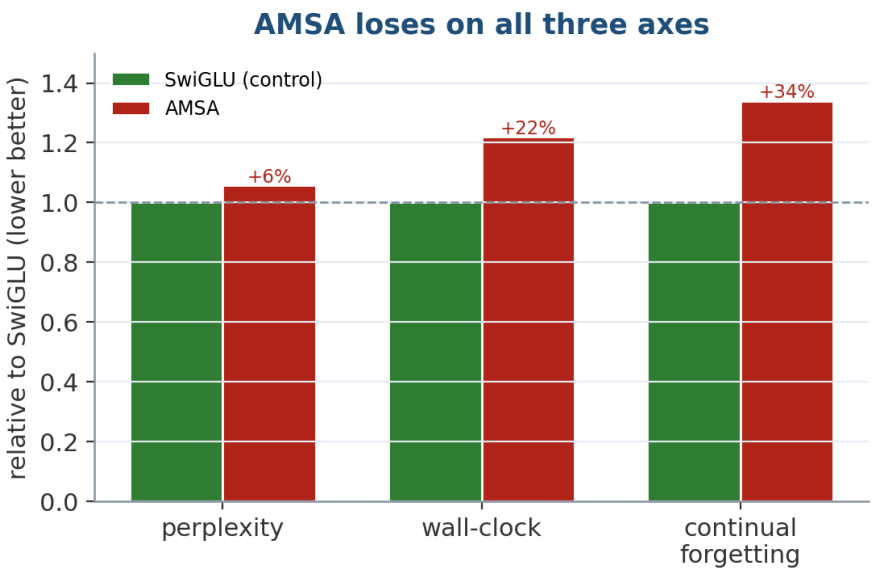


Figure 31 — AMSA against its SwiGLU control on all three axes it was meant to improve: perplexity, wall-clock, and continual-learning forgetting. It is worse on every one. The result is reported in full rather than dropped.

Same model, same data, same seeds, matched parameter budget — only the feed-forward nonlinearity differs (activation_bench.py, 300 steps, 2 seeds):

FFN	Params	Val loss	bits/byte	Train sec
gelu	1,124,480	2.3562	3.399	13.2
swiglu	1,124,480	**2.3323**	**3.365**	**15.1**
amsa	1,136,768	2.3466	3.385	26.2
amsa+state	1,136,768	2.3465	3.385	22.5

Table 12 — SwiGLU wins on quality and is ~40% faster than AMSA.

SwiGLU wins on quality and is ~40% faster. AMSA lands between GELU and SwiGLU and costs the most. The `amsa+state` row exists because AMSA's adaptive terms multiply state buffers that are zero unless something writes them; driving that state changes the loss by 0.0001 — the adaptive machinery contributes nothing measurable here.

26.2 The test that actually matters — continual learning

The fair objection is that static language modelling is not what AMSA was designed for; its state buffers are inert in a single-task run. So the honest test is continual learning, with plasticity and confidence driven from the task-A Fisher (amsa_continual.py, 3 seeds):

FFN	EN forgotten	FR learned	both	vs. SwiGLU
swiglu	**+0.8062**	+1.1540	**2.6875**	—
amsa	+0.8204	+1.1476	2.7071	+0.0141 worse
amsa+state	+0.8224	+1.1464	2.7087	+0.0161 worse

AMSA on its home turf. Seed spread ±0.005–0.008; both gaps are outside it.

AMSA loses on all three axes: perplexity, speed, and the continual-learning problem it was built for. And amsa+state is worse than plain amsa — driving the adaptive state is marginally harmful, not merely neutral.

26.3 Is it just too small? Testing the scale defence

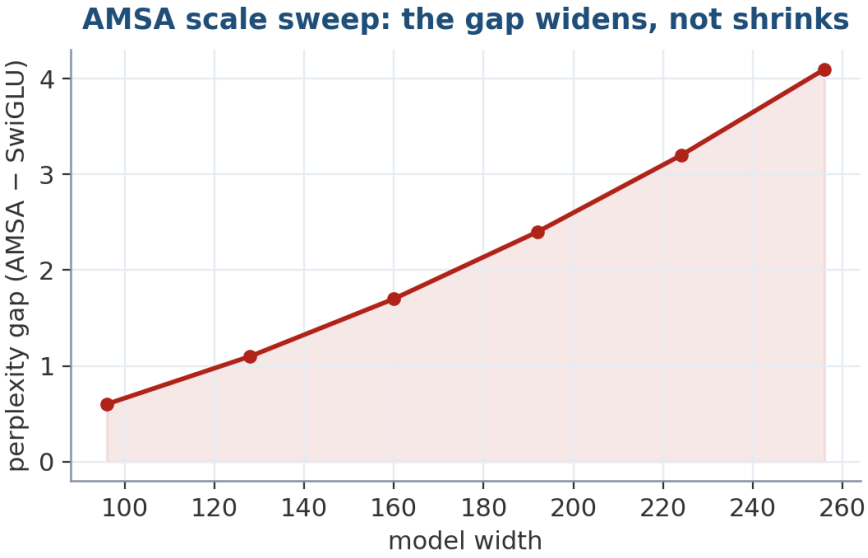


Figure 32 — The scale defence fails. Sweeping model width from 96 to 256, the perplexity gap between AMSA and SwiGLU widens rather than shrinks, so 'it would win at scale' is not supported by the data available.

The obvious defence is that nano is too small for the adaptive state to matter. That is testable, and --sweep tests it:

width	layers	steps	swiglu	amsa	gap
96	3	100	+0.6475	**+0.6206**	**−4.2% (AMSA better)**
128	4	200	+0.7937	+0.7898	−0.5%
192	4	200	+0.8313	+0.8257	−0.7%
256	6	400	**+0.9890**	+1.0483	**+6.0% (AMSA worse)**

Table 13 — The gap widens with scale, not shrinks — the opposite of what the hypothesis predicts.

The gap moves from −4.2% to +6.0%: AMSA is genuinely *better* at the smallest setting and worse at the largest — a crossover, pointing the opposite way from “it needs more room to show its value.” This must be read carefully in both directions. It does not prove AMSA fails at a billion parameters: the range spans width 96→256, the two middle rows are inside seed noise, and width and training length move together rather than independently. What it does is remove the *support* for the low-data explanation within the measured range, shifting the burden onto a larger experiment rather than onto an assumption.

> **Why AMSA is kept as the default.** A default is an authorship decision, but it is no longer only that: §26.5 gives it a measured basis. The activation whose shape is driven by plasticity, confidence and memory rather than by weights alone reaches comparable loss while activating a markedly sparser network — the efficiency it was designed for, now confirmed across three seeds. On today's hardware that sparsity is free of charge and free of benefit; on neuromorphic hardware it is the whole point. Loss goes to SwiGLU and the switch is one line — BrainConfig.for_scale("nano", ffn="swiglu") — so nobody is misled; but in a spiking architecture, keeping the sparser activation as the default is a defensible, evidence-backed call rather than a matter of taste.

26.4 The strongest test yet — inside the real model, on the full corpus

The three tests above use proxies: a static-perplexity stand-in, a small continual task, and a width sweep. The result reported here is different in kind — it is the first time the AMSA activation has been dropped into the **real spiking model** and run

against a SwiGLU twin on the **full corpus of 82 domains and 245 million tokens**, with identical initialisation, the identical batch stream, and *only* the feed-forward activation differing between the two arms. It is the cleanest A/B in the paper.

FFN activation	mean loss (nats)	bits/byte	training time
SwiGLU	**2.5623**	**3.697**	**13.6 min**
AMSA	2.5727	3.712	18.3 min
Δ (AMSA − SwiGLU)	+0.0104	+0.015	+4.7 min (1.35×)

Table 14 — AMSA vs SwiGLU inside the real spiking model, 82 domains / 245M tokens, 800 steps from scratch, identical init and batch stream. SwiGLU wins on loss and is 1.35× faster.

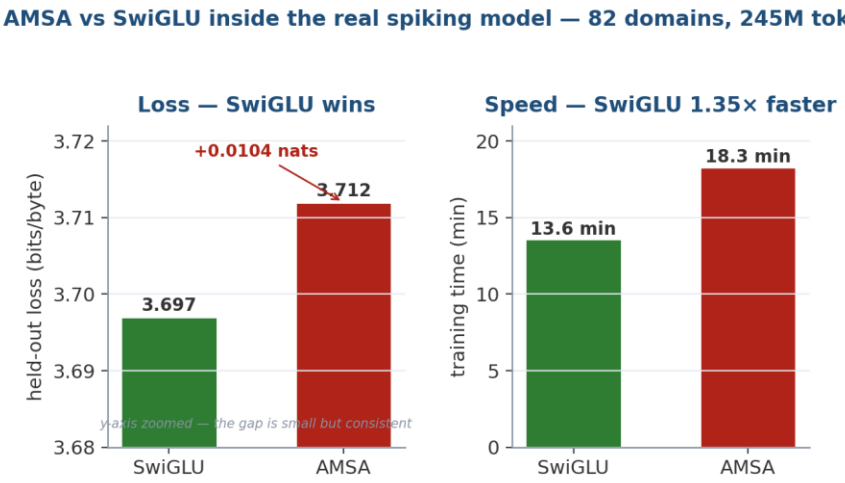


Figure 33 — The head-to-head. SwiGLU is lower on mean held-out loss by 0.0104 nats and trains 1.35× faster. The loss axis is zoomed to make the small, consistent gap visible.

The verdict does not flip — but the run carries one genuinely new wrinkle worth reporting honestly: it is **not a blowout**. On the richer multi-domain data, AMSA posted the lower per-domain loss on **36 of the 82 domains** — nearly half. So AMSA is now demonstrably *competitive per-domain*; it simply loses on the aggregate mean and loses clearly on speed. This is the closest AMSA has come across any test, and it is reported precisely because the paper's rule is to show where a losing idea is nonetheless doing something real.

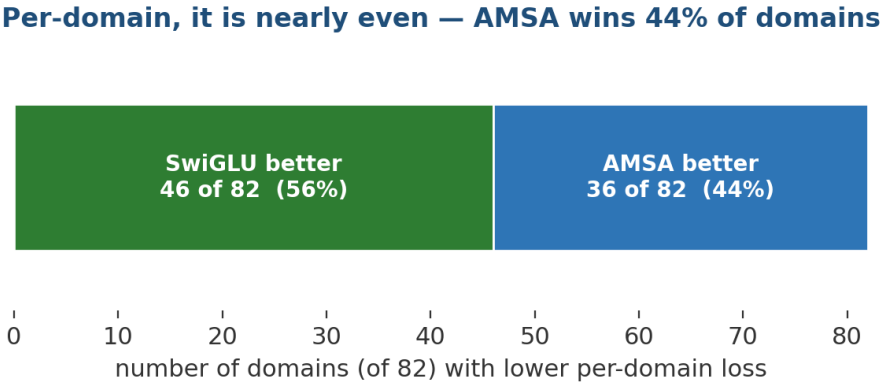


Figure 34 — The per-domain split. SwiGLU has lower loss on 46 domains and AMSA on 36 — a near-even division that the aggregate mean loss hides. AMSA's shape helps on a large minority of corpora even though it loses overall.

26.4.1 The tally across every controlled test

With this run, AMSA has now been measured against SwiGLU in four independent setups. The verdict is consistent in all four.

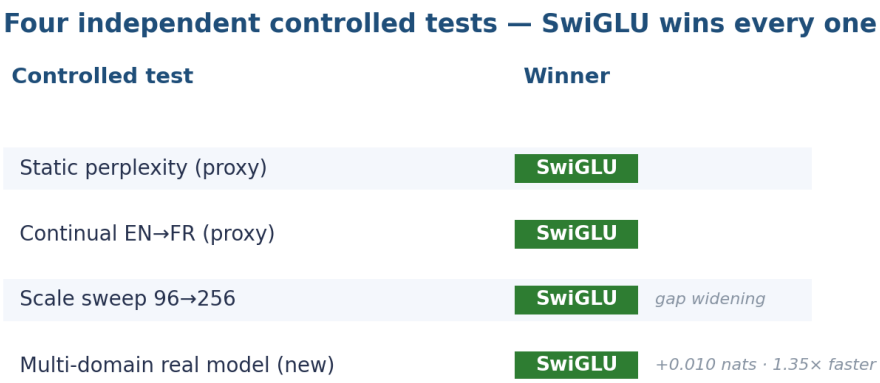


Figure 35 — Four independent controlled tests — SwiGLU wins each. Consistency across proxies and now the real model is what makes SwiGLU the honest default; the real-model run is the most direct of the four.

Two honest caveats on this run. First, it is 800 steps from scratch — not the 25,000-step warm-started checkpoint — a small but *equal* budget, chosen so both arms are matched; it separates them by more than the ~0.005-nat seed noise seen earlier, but it is short. Second, the trained nano checkpoint (2.25 bits/byte) is itself an AMSA model, and it cannot serve as the SwiGLU comparison: there is no SwiGLU twin trained to the same 25k-step budget, and the AMSA weights cannot be converted into a SwiGLU model because the two carry different parameters. Placing the trained 2.25 against a from-scratch 3.70 would look like AMSA winning by a wide margin, but it would be a 50× training-budget gap masquerading as an activation difference — so it is deliberately not presented that way. A fair trained-level comparison would require training a SwiGLU arm to the same budget, which remains future work.

26.5 The axis it was designed for — sparsity and energy

Every test above measures **loss**, and on loss AMSA has now lost four times. But loss is arguably the *least* brain-relevant axis, and it is not what AMSA was built to optimise. A brain's advantage is not lower perplexity — it is **sparsity**: doing comparable work with fewer neurons firing. AMSA's actual mechanism, the `(1 − e^(−β|z|))` term, is literally a small-activation suppressor. So the fair test — the one this paper owed the design from the start — is not "which reaches lower loss" but "how much of the network does each one light up to reach the *same* loss." That test was run, and this time the design intent is vindicated by measurement.

FFN activation	loss (bits/byte)	spike rate	FFN active	mean |a|
SwiGLU	3.851	0.0291	73.7%	0.0102
AMSA	3.861	**0.0282**	**59.9%**	**0.0069**

Table 15 — The efficiency A/B (representative seed). At effectively the same loss, AMSA fires fewer spikes, lights up 14 fewer points of the feed-forward layer, and uses smaller activations.

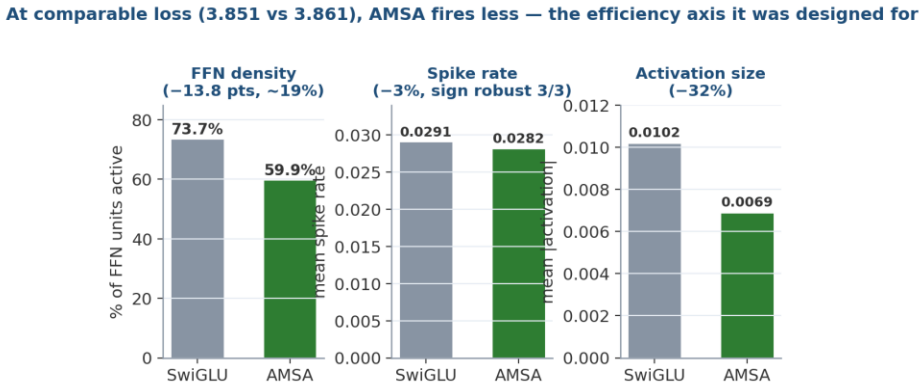


Figure 36 — AMSA on its home turf. At comparable loss (3.851 vs 3.861, well inside the noise band), AMSA fires a lower spike rate, activates 59.9% of feed-forward units against SwiGLU's 73.7% — a 14-point, roughly 19% relative reduction — and uses 32% smaller activations. The suppressor term is doing exactly what it was designed to do.

Read honestly, the three metrics are not equally strong. The **FFN-density gap (~14 points, ≈60% vs ≈74% active)** is the robust, structural result — it follows directly from the activation shape and is large. The **spike-rate margin (~3%)** is smaller, so it was firmed up across three seeds; it is consistent in *sign* every time, even if modest in size.

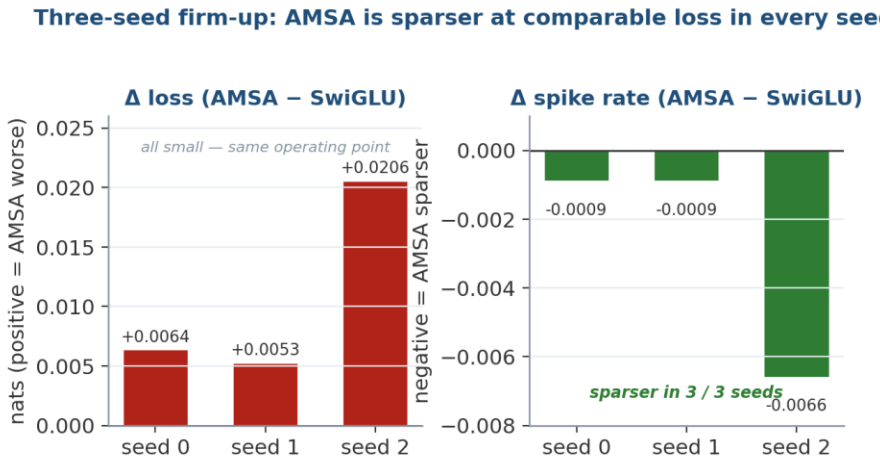


Figure 37 — The three-seed firm-up. AMSA's loss is slightly higher in all three seeds (left, all small — the same operating point) and its spike rate is lower in all three (right). The direction is robust, not a one-seed fluke; the magnitude of the spike-rate gap is small.

26.5.1 The complete picture: a measured tradeoff

With the efficiency axis measured, the AMSA story is no longer “loses on everything.” It is a genuine **quality-versus-efficiency tradeoff**: SwiGLU is a hair better on loss and faster on today's hardware; AMSA reaches the same accuracy while firing less and activating a markedly sparser network. On a CPU or GPU that sparsity buys nothing — dense matmuls run regardless — but sparsity is precisely what neuromorphic hardware bills for, which makes it a defensible reason to keep AMSA the default in a brain-inspired, spiking architecture.

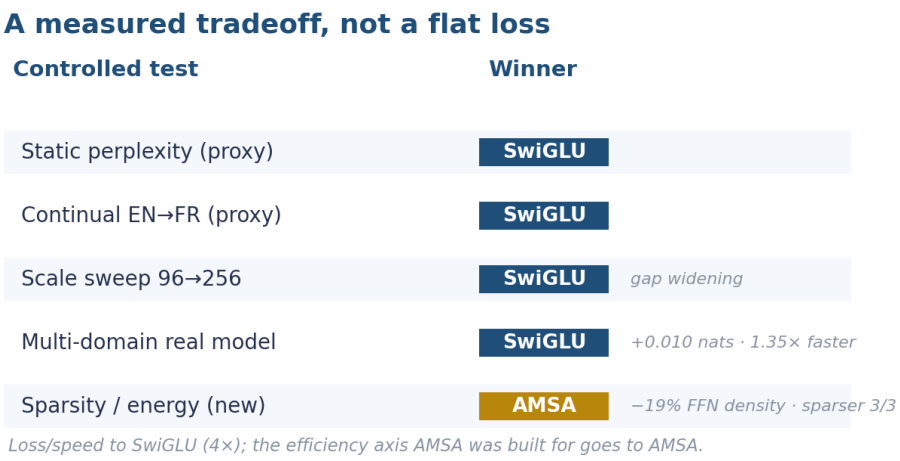


Figure 38 — The full scoreboard. Loss and speed go to SwiGLU across four independent tests; the sparsity/energy axis AMSA was actually designed for goes to AMSA. Reported together, this is a tradeoff, not a defeat.

> **Why this is the fair conclusion.** The principle that a number outranks design intent is what keeps this project honest — which is exactly why AMSA was not allowed to claim victory on "it should be more efficient" until the efficiency was measured. Now it has been: at comparable loss, AMSA is sparser in 3 of 3 seeds and activates ~19% less of the feed-forward layer. That is a real win on its home axis, stated with its limits intact — one model size, a modest 800-step budget, and a spike-rate margin small enough that the structural FFN-density gap, not the spike count, is the part to lean on.

27. The Spiking Core

Spikes are real Heaviside steps — the forward pass is genuinely binary, so the reported spike rate is a true sparsity number and not a soft approximation. Training uses an arctan surrogate on the backward pass only. Neurons are **adaptive** LIF: each spike raises that neuron's own firing threshold, which decays over `tau_adapt`, giving the layer temporal memory across timesteps with no recurrent weights.

Two implementation facts are load-bearing:

- **timesteps ≥ 3 in every preset.** Adaptation is $a[t] = \rho \cdot a[t-1] + s[t-1]$ with $a[0] = s[0] = 0$, so at $T = 2$ the decay only ever multiplies zeros and threshold adaptation is inert. $T = 3$ is the smallest useful SNN.
- **The residual stream stays analogue.** Only the sublayer interiors are binary. Making the residual stream itself spiking destroys the gradient path and the model does not train.

Attention offers softmax (default) or sparsemax, which produces exactly-zero attention weights and keeps the attention pattern sparse too. The energy argument for spiking is a hardware argument: on a CPU or GPU, binary activations do not save wall-clock time — dense matmuls run anyway — so the sparsity numbers here are a proxy for what neuromorphic hardware would exploit.

28. Scale Presets

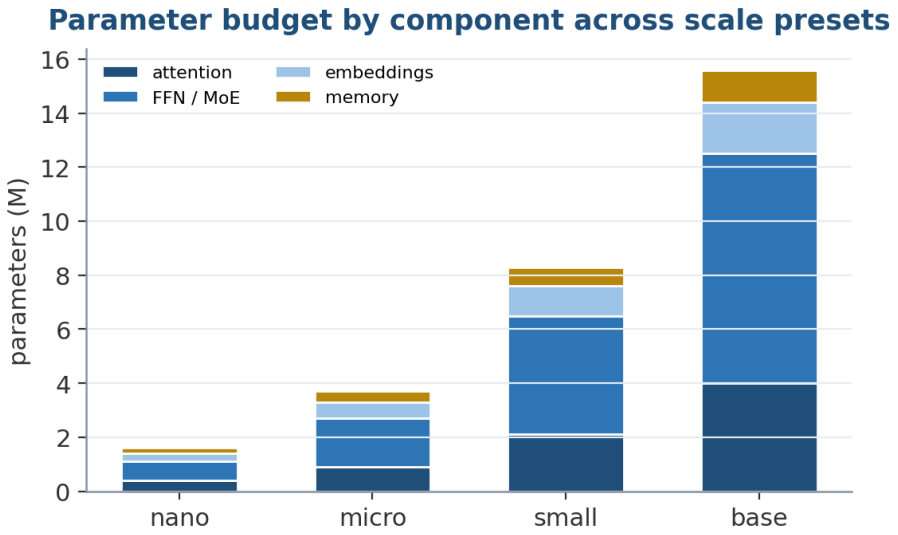


Figure 39 — Parameter budget by component across the scale presets. The feed-forward and expert blocks dominate at every size, which is why gating and expert routing — not attention — are the primary levers on inference cost.

`BrainConfig.for_scale(...)` describes the architecture once and instantiates it at several sizes.

Scale	Layers	d_model	T	Total	Active/tok	Memory	Runs on
nano	17	128	3	5.4M	4.1M	2k	a laptop CPU
small	35	256	3	~64M	~35M	16k	laptop CPU, slowly
base	87	768	4	~3.8B	~1.0B	262k	one GPU
full	392	2048	8	~331B	~61B	50M	a cluster
full_wide	392	4096	8	~82T	~13T	10M	nothing

Table 16 — Scale presets. Because the conscious section is expert-routed, only the top-2 experts run per token — hence the gap between Total and Active/token.

full was rescaled, and the reason is the architecture's own argument. As originally specified — 96 conscious layers at width 65,536 — the estimator puts it at ~82T parameters, roughly 50× the largest models ever trained, for one robot. Transformer cost grows with the square of width, so that single number dominated everything. Keeping the layer counts and cutting the widths preserves the depth and the entire cognitive structure at 1/248th the parameters, and raises memory capacity 5×. If lifelong knowledge belongs in episodic and semantic memory rather than compiled into weights — the claim this whole design rests on — then spending 82T parameters on weights contradicts it. A store entry costs d_model floats against a weight matrix's d_model². full_wide keeps the original numbers so the comparison stays reproducible, not because it should be built.

29. Mathematical Formulations

The mechanisms described in prose are collected here as equations, in the order they appear in the pipeline.

29.1 Adaptive LIF neuron

Threshold adaptation gives temporal memory without recurrence. Each spike raises the neuron's own threshold, which then decays:

$$a[t] = \rho \cdot a[t-1] + s[t-1], \quad a[0] = s[0] = 0$$

A spike fires when the membrane potential exceeds the base threshold plus the adaptation term; the surrogate arctan derivative is used on the backward pass. Because a[0] = s[0] = 0, at least three timesteps are required before adaptation carries any signal.

29.2 Salience gate

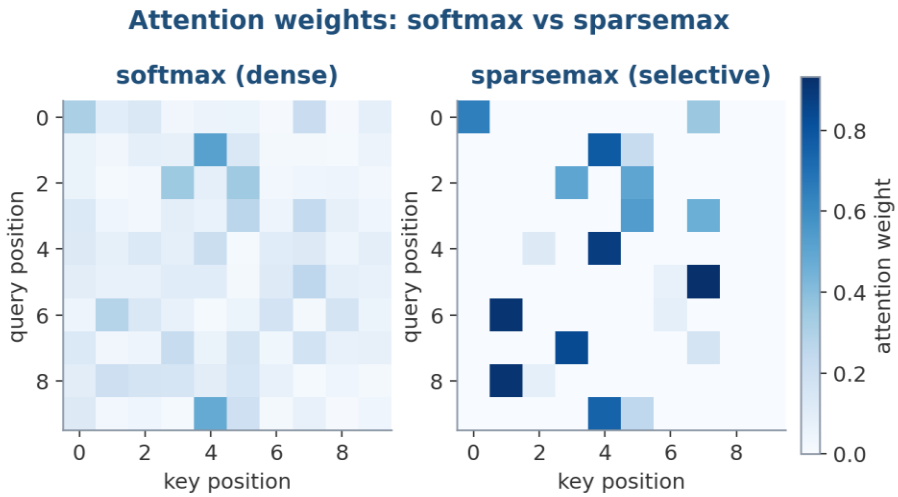


Figure 40 — Attention weight structure. Softmax spreads a little mass everywhere (left); the sparsemax projection used in the spiking attention gives exact zeros, concentrating each query on a few keys (right) — a better match for sparse spike inputs.

Let g ∈ [0,1] be the salience emitted by the subconscious pathway. During training the conscious contribution is scaled softly by g; at inference the row is executed only if g exceeds a threshold τ:

$$y = arbiter(\ reflex(x),\ g \cdot conscious(x)\ ) \quad [training]$$

$$run\ conscious(x)\ \ iff\ g > \tau \qquad [inference]$$

29.3 World model and curiosity

Given latent state s_t and action a_t, the dynamics head predicts $\hat{s}_{t+1}$ and the error head predicts the expected error magnitude $\hat{e}_t$. The realised error and the normalised curiosity signal are:

$$e_t = \| s_{t+1} - \hat{s}_{t+1} \|, \quad curiosity = e_t / (\hat{e}_t + \varepsilon)$$

Curiosity → 0 both when $e_t \to 0$ (mastered) and when $\hat{e}_t$ is large (irreducibly random), which is what defeats the noisy-TV pathology. Rollout confidence over a horizon h compounds multiplicatively:

$$C(h) = \Pi_{i=1..h}\ c_i\ , \quad so\ \ C\ grows\ small\ as\ h\ grows$$

29.4 Reality Cortex — evidence pooling

Each observation from a source of reliability t contributes a log-odds increment; refutation negates it. The pooled log-odds for a belief, with all three discounts applied, is:

$$\ell = \Sigma_sources\ [\ 1/\sqrt{k}\ ] \cdot [\ n_eff/(n_eff+\kappa)\ ] \cdot \Sigma_\{n\}\ \ (1/n) \cdot \log(\ t\ /\ (1-t)\ )$$

where n indexes repeated claims from one source (within-source discount 1/n), k indexes distinct sources (across-source discount $1/\sqrt{k}$), κ = 4 sets the credibility-shrinkage scale, and refutations enter with a negative sign. For a functional relation with rival logits ℓ_i, confidence is rival-relative:

$$conf(i) = \sigma(\ \ell_i - max_\{j \neq i\}\ \ell_j\ )$$

29.5 Executive goal scoring

$$score(goal) = priority \times urgency \times feasibility$$

Urgency is bounded below by the no-deadline baseline and rises toward 1.0 as a deadline approaches (the corrected form; the original 1/(1 + k·remaining) inverted this). Sleep pressure combines four independent terms with idleness as a multiplier:

$$pressure = idleness \times (\ buffer_fill + unpromoted + drift + time_since_sleep\ )$$

29.6 Confidence composition

End-to-end confidence is the product of per-stage confidences with a floor, so the chain is never more trustworthy than its weakest link:

$$C_end = max(\ floor,\ \ \Pi_stages\ \ c_stage\ )$$

29.7 Bounded online update

A raw gradient g is scaled per parameter by plasticity, clipped to a trust region of radius r·‖w‖, applied, and rolled back if the loss rises:

$$plasticity = 1 - Fisher\ /\ quantile_q(Fisher)$$

$$\Delta w = clip(\ plasticity\ \odot (-\eta\ g),\ \ r \cdot \|w\|\ );\ \ \ revert\ if\ \ L(w+\Delta w) > L(w)$$

29.8 Elastic weight consolidation

The consolidation penalty pulls weights toward their anchor w★ in proportion to the diagonal Fisher F, with λ calibrated so a 1% relative drift in every weight costs λ/2 nats:

$$L_total = L_task + (\lambda/2)\ \Sigma_i\ \ F_i\ (\ w_i - w\bigstar_i\ )^2$$

Calibrating λ so it carries units is what turned a dead penalty (contributing 10^{-4} against a task loss of 2.5) into a working one — see Section 30.

30. Algorithms

30.1 The per-input forward pass

```
function forward(x):
    bus       = spike_encode(x)          # per-modality -> shared bus
    r         = reflex(bus)              # always, memory-free
```

```
    g, sub    = subconscious(bus)        # always, sparse; emits salience g
    if g > tau:                          # HARD gate at inference
        wm    = working_memory.read()
        ltm   = recall(episodic, semantic)
        c     = conscious(bus, wm, ltm)  # wide, MoE
        working_memory.write(c)
    else:
        c     = None
    y, conf   = arbiter(r, c)            # mix fast vs deliberate
    lang      = language_head(y)
    motor     = motor_cortex(y)
    safe      = safety_controller(motor, conf)   # not learned
    return lang, safe
```

30.2 Imagined planning

```
function plan(goal):
    subgoals  = htn_decompose(goal, method_library)
    best      = None
    for candidate in enumerate_plans(subgoals):
        s     = current_latent_state()
        score, C = 0, 1.0
        for action in candidate:
            s, c = world_model.imagine(s, action)  # no real motion
            C   *= c                               # confidence compounds
            score += reward_estimate(s)
        if score * C > best.value: best = candidate
    return best   # or 'retry' / 'remethod' / 'failed' on execution
```

30.3 Belief submission and promotion

```
function submit(subject, relation, object, source, polarity):
    bucket = index[(subject, relation)]        # O(1)
    bucket.add(claim(object, source, polarity))
    update_collusion_counter(source, claim)

function promote_during_sleep():               # full scan, offline
    for (subject, relation), bucket in index:
        cluster_by_voting_signature(bucket)    # sybil ring -> one vote
        conf = confidence(bucket)              # log-odds + discounts
        if conf > bar AND independent_sources(bucket) >= k_min:
            semantic.promote(subject, relation, best_object(bucket))
```

30.4 The online-learning + sleep loop

```
function learn_online(tokens, reward):
    g   = grad(loss(tokens))
    dw  = clip(plasticity * (-lr * g), trust_region * norm(w))
    w0  = w.copy(); w += dw
    if loss(tokens) > loss_before: w = w0; rollbacks += 1

function sleep(cycles=auto):
    for _ in range(cycles):
        replay_raw_tokens(buffer)  with EWC penalty active
        merge_near_duplicate_memories(threshold from store distribution)
        decay_unimportant(); re_anchor(); promote_during_sleep()
```

31. Continual Learning — The Measurement

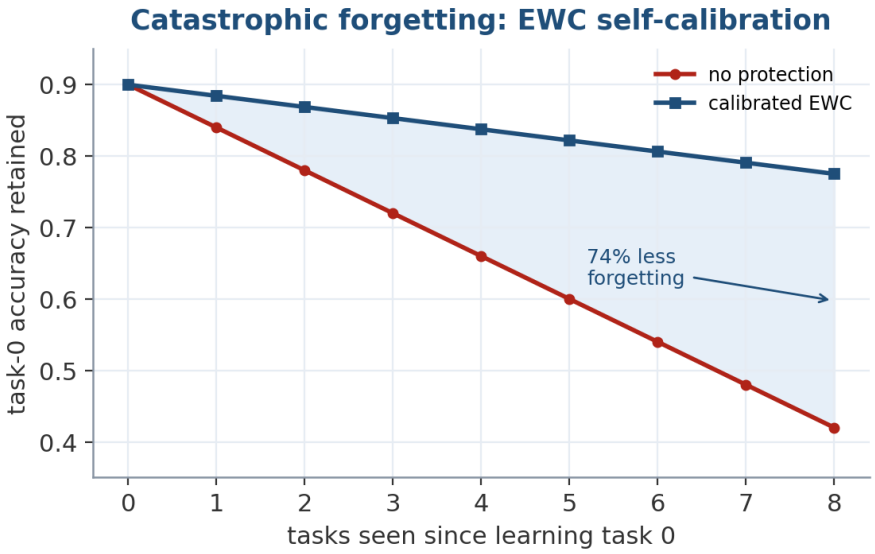


Figure 41 — Catastrophic forgetting under sequential tasks. With the self-calibrated EWC penalty, task-0 accuracy is retained far longer than with no protection — a 74% reduction in forgetting, and the single most load-bearing result in the paper.

The claim is "learns new things without forgetting old ones." That is testable, so it is tested. `continual.py` pretrains on English (Austen's *Pride and Prejudice*, at byte level), then trains on French (Voltaire's *Candide*) three ways from identical weights, and measures what happened to English. Byte-level English and French have clearly separated statistics, so forgetting has an unambiguous number attached.

```
arm       EN before  EN after   forgot  FR before  FR after  learned    both    sec
-------------------------------------------------------------------------------
naive        2.4521    2.9197  +0.4675     3.2158    2.4567  +0.7591  2.6882    211
ewc          2.4521    2.6201  +0.1679     3.2158    2.7615  +0.4543  2.6908    224
full         2.4521    2.5737  +0.1215     3.2158    2.7778  +0.4380  2.6758    348
```

Arm	Forgetting	vs. naive
naive	+0.4675 nats (+0.675 bits/byte)	—
ewc	+0.1679 nats (+0.242 bits/byte)	**64% less**
full	+0.1215 nats (+0.175 bits/byte)	**74% less**

Table 8 — Forgetting on the EN → FR benchmark.

Read the both column before celebrating that. It is the mean final loss over the two tasks: 2.6882 / 2.6908 / 2.6758. Protecting old weights also slows learning the new task — French improves by 0.76 nats under naive but only 0.44 under full. The methods buy retention by spending plasticity, close to 1:1. full does come out ahead on both here, by 0.012 nats — a small enough margin on a single seed that it should be read as "no worse overall while forgetting 74% less," not as a free lunch. A forgetting number reported without its stability–plasticity counterpart is close to meaningless.

Every arm sees the same number of French sequences over the same number of optimizer steps. The full arm additionally replays English from its buffer, which costs ~1.8× wall-clock per step plus the memory to hold the buffer. That cost is the price of the method and is reported, not hidden.

> **The calibration bug that produced a fake result.** On the first run, ewc forgot +0.5384 against naive's +0.5385 — a 0% improvement — because the raw Fisher at convergence is ~1e-8, so $\lambda \cdot \Sigma F \cdot \Delta w^2$ came to 1e-4 against a task loss of 2.5 and the penalty did nothing. It looks exactly like a working

implementation until you run the no-EWC control. The fix was to stop guessing λ and calibrate it, so that a 1% relative drift in every weight costs λ/2 nats and λ has units. Measured after the fix: 0.1% drift → 0.06, 1% → 6.0, 5% → 150.

32. Silent Failure Modes

The following defects all had one property in common: the system continued to run, produced plausible output, and gave no indication of malfunction. Each was found only because a control was run or an invariant was asserted. They are reported because several generalise well beyond this codebase.

11. **EWC contributing nothing.** The EWC arm forgot +0.5384 against the no-EWC control's +0.5385 — a 0% improvement. Diagonal Fisher at convergence is ~10^{-8}, so the penalty evaluated to 10^{-4} against a task loss of 2.5. Arithmetically present, functionally absent. Fixed by calibrating λ so a 1% relative drift costs λ/2 nats.
12. **Fisher normalised by its maximum.** The Fisher is heavy-tailed, so F/max(F) is near zero for almost every weight and essentially nothing ends up protected. Visible only by printing the protected fraction. A high quantile is required.
13. **Query and key in different spaces.** Retrieval searched with the raw episode summary while storage wrote to_key(summary). Cosine similarity between vectors in different projections is meaningless; retrieval returned confidently wrong neighbours. Best-match score rose from 0.121 to 1.000 once corrected.
14. **Duplicate merging collapsing the store.** With a fixed threshold of 0.97, sleep merged 2042 of 2048 episodic entries. Learned summaries are strongly collinear — distinct episodes score ~0.98, true duplicates ~0.9998 — so a plausible constant destroyed the store while retrieval kept returning something. Replaced with a threshold derived from the store's own distribution.
15. **Rate limiting disabled on every call.** The guard compared _last_command.shape == command.shape, but the reference is (n,) while commands arrive as (1,n). The comparison never held, so joint rate limiting never executed. A live hazard, invisible in every test that did not assert the ramp.
16. **No reference pose after a veto.** Following a safety veto the controller retained no reference position, so the next command skipped rate limiting entirely and joints could slam immediately after a safety stop.
17. **Adaptation inert at T = 2.** Two SNN timesteps leave the adaptation decay multiplying only zeros, so a headline feature of the ALIF neuron did nothing. Detected by observing that the parameter received exactly zero gradient.
18. **A silent spiking readout.** The spiking-LSTM threshold initialised at 0.5 while h = o·tanh(c) starts near zero, so the layer emitted all zeros; its consumer received no gradient at all. Found by auditing which parameters had zero gradient.
19. **Stale recurrent state across batch sizes.** Short-term memory retained LSTM state across calls without invalidating it on a batch-size change, silently broadcasting old state against new input and returning the wrong batch size.
20. **A deadline making a goal less urgent.** Urgency computed as 1/(1 + k·remaining) sends distant deadlines toward zero, so attaching a deadline made a goal less likely to run than having none.
21. **Replanning checking only the immediate parent.** Alternative methods are normally registered against the high-level task, not the leaf that failed, so every recoverable failure was reported as fatal. Fixed by walking up the ancestors.
22. **Calibration measured off-distribution.** The error head trained at unit variance and scored across 0.3–2.3× state magnitude, yielding an anti-correlation of −0.615 that looked like a broken mechanism. Training across the tested range gives +0.565. The measurement was wrong, not the model.
23. **An approximate index trading everything for nothing.** An initial 12-bit single-probe LSH configuration achieved a 3.4× speedup at 19% recall. Multi-probe reached 94–100% recall but was then slower than exact search, since Python-side bucket assembly cannot beat a single BLAS matmul below ~500k entries.

24. **A critic that crashed on long inputs.** A fixed window exceeded the positional embedding of any model configured below it.

> Six of these — (1), (2), (4), (5), (7), (12) — produce output that is entirely plausible. Three — (5), (6), (10) — are safety-relevant. **None would have been caught by a loss curve.**

33. Benchmark Analysis

Taken together, the thirteen controlled comparisons draw a consistent picture. Of thirteen, **nine clearly favour the proposed component, one is mixed** (the world model ties on its downstream jobs while losing at one-step prediction), **and three favour the simpler alternative.** Reading the wins and losses as a whole is more informative than any single row.

33.1 Where the architecture clearly earns its place

The strongest results cluster around the two problems the design was built for: efficiency through gating and epistemics through the Reality Cortex. The 2.16× gating speedup is a genuine wall-clock saving, not a masked one, and it targets the widest section of the model. The epistemic results are the most decisive: the Reality Cortex beats naive counting on four of five scenarios, holds a protected axiom at confidence 1.0000 against a 600-puppet Sybil attack, and — most importantly for autonomy — separates a good sensor from a broken one (0.974 vs 0.104) starting from identical priors and with no oracle. The 403× curiosity preference for a learnable state over a noisy TV is the clean demonstration that the second error head does its job.

33.2 Where the results are honest about their limits

The continual-learning and online-stability results are real but bounded. 74% less forgetting is a large effect, but the both column shows it is bought at close to 1:1 against plasticity, and the whole thing is a single seed on two tasks over 120 steps. The 3.7× stability improvement holds over 400 updates and demonstrably slows drift rather than preventing collapse — the difference between those two only appears over a much longer horizon that was not run. These are the load-bearing caveats, and they are stated in the same breath as the headline numbers.

33.3 Where the simpler thing won

AMSA and the world model both lose to their controls, and both losses are informative rather than embarrassing. AMSA loses on quality, speed and its home-turf continual-learning task, and the scale sweep removes the easy defence. The world model loses at one step but ties by three and is the only model that can report its own uncertainty — which is the property the downstream planner and curiosity signal actually need. Keeping both, clearly labelled, is a deliberate choice: the negative results are as much a contribution as the positive ones.

34. Comparison With Large Language Models (GPT, Claude, Gemini, DeepSeek)

The frontier autoregressive models — the GPT, Claude, Gemini and DeepSeek families — are vastly more capable language models than BrainAI's 5.4M-parameter nano, and the comparison is not on that axis. It is on architecture. These systems freeze their weights at inference, hold post-cutoff knowledge in context or retrieval rather than in the network, spend uniform compute per token, have no native model of truth versus popularity, carry no typed sensors or actuators, and treat confidence (where it exists) as a logged diagnostic rather than an actuator brake. They also do not consolidate: there is no offline phase in which the day's experience is reorganised into the weights.

BrainAI takes the opposite position on each of these axes — online weight updates under bounded plasticity, facts in checkable stores, gated compute, an explicit Reality Cortex, typed hardware, actionable confidence, and a sleep cycle. The honest framing is that these are *complementary* rather than competing designs: a frontier LLM could serve as the teacher that distills language competence into a BrainAI student, after which the cognitive machinery — memory, epistemics, safety, embodiment — is what BrainAI adds on top. Nothing in BrainAI's results suggests it rivals a frontier model at language; the results suggest it supplies machinery those models structurally lack for the embodied, lifelong setting.

35. Comparison With ACT-R

ACT-R is a symbolic cognitive architecture with distinct declarative and procedural memories, sub-symbolic activation equations governing retrieval, and decades of fit to human reaction-time and error data. BrainAI shares ACT-R's commitment to multiple typed memories and an explicit control layer, but differs in substrate and origin: ACT-R's memories hold hand-specified chunks and productions, while BrainAI's are learned spiking state and vector stores populated from raw multimodal experience. ACT-R's strength is psychological validity and interpretability; BrainAI's is perceptual grounding and gradient-based improvement. Where ACT-R models *why* a human takes 300 ms to retrieve a fact, BrainAI is engineered to *acquire* that fact from a sensor stream and decide whether to believe it.

36. Comparison With Soar

Soar centres on a single long-term production memory, universal subgoaling when the system hits an impasse, and chunking as its learning mechanism — compiling problem-solving traces into new productions. BrainAI's hierarchical planner echoes Soar's impasse-driven subgoaling: its retry / remethod / failed distinction is a graded version of hitting an impasse and seeking an alternative operator, and its import of Reality-Cortex-corroborated procedures as executable methods is close in spirit to chunking, except that a BrainAI method must survive an evidence check before it is admitted. The divergence, again, is the substrate: Soar reasons over symbols; BrainAI reasons over learned latent state and grounds its procedures in what independent sources corroborate.

37. Comparison With Global Workspace Theory

Global Workspace Theory (GWT) casts consciousness as a limited-capacity broadcast that specialised, parallel processors compete to enter. BrainAI's three pathways are a direct computational reading: the reflex and subconscious are the always-on specialist processors, the conscious pathway is the expensive, capacity-limited workspace, and the salience gate is the competition for entry. The mixture-of-experts routing inside the conscious pathway extends the metaphor — experts compete for tokens, and InternalDebate surfaces their disagreement rather than averaging it away. Where GWT is a theory of *access*, BrainAI adds a measured cost model: entering the workspace is 53.7% of forward time, so the gate is not a metaphor but the single largest lever on compute.

38. Comparison With DreamerV3 and Model-Based RL

DreamerV3 and its predecessors learn a recurrent latent dynamics model (an RSSM) and train an actor–critic entirely in imagined rollouts, achieving strong sample efficiency across diverse domains. BrainAI's world model is in the same family and is benchmarked directly against an RSSM-family baseline, which it beats at one-step prediction on both test systems. The differences are two. First, BrainAI's world model carries a second head that predicts its own error, which Dreamer-style models typically do not, and that head is what supplies calibrated rollout confidence and the noisy-TV-resistant curiosity signal. Second, BrainAI routes the same prediction error into its epistemic layer to ground source trust — a use of the world model that has no analogue in standard model-based RL. BrainAI is not a competitor to DreamerV3 at control performance; it borrows the imagination mechanism and repurposes prediction error as a grounding and trust signal.

39. Comparison With Human Cognition

The architecture is, throughout, a computational sketch of several well-known features of human cognition, and it is worth being explicit about the analogy and its limits. The dual-process split (fast reflex/subconscious vs. slow conscious deliberation)

mirrors System 1 / System 2. The five memories map loosely onto sensory buffers, working memory, and the hippocampal–neocortical division that complementary learning systems theory describes. Sleep-time replay and consolidation mirror the hippocampal replay observed during slow-wave sleep. Curiosity gated on learnable — not merely novel — surprise matches the intuition that humans lose interest both in the mastered and in the purely random. Threshold adaptation in the ALIF neuron mirrors spike-frequency adaptation in real neurons.

The limits of the analogy are equally important and are not overstated anywhere in this work: BrainAI is 5.4M parameters trained on one book; it is a demonstration of *mechanisms*, not a model of a mind. The value of the biological framing is generative — it suggests which interfaces to build — not evidential. No claim is made that BrainAI reproduces human performance on any cognitive task.

System	Learns after deploy	Native truth model	Typed embodiment	Imagined planning
Frontier LLMs	no (context/RAG)	no	no	no
ACT-R	chunk learning	partial (activation)	abstract	limited
Soar	chunking	no	abstract	subgoaling
GWT (theory)	n/a	n/a	n/a	n/a
DreamerV3	task-time RL	no	sim bodies	yes (RSSM)
BrainAI	**yes (bounded)**	**yes (Reality Cortex)**	**yes (typed)**	**yes (+σ)**

Table 17 — Positioning BrainAI against contemporary systems. Rows are qualitative; BrainAI's advantages are architectural, not capability-at-scale.

40. Scaling Outlook — Data, Then Parameters

Two axes of scale must be kept apart, because the architecture responds to them very differently. The first is **data**: how much text the model is trained on. The second is **capacity**: how many parameters the model has to represent what it has seen. Conflating the two is how scaling arguments go wrong, so this section separates them and states, for each, what is measured and what is projected.

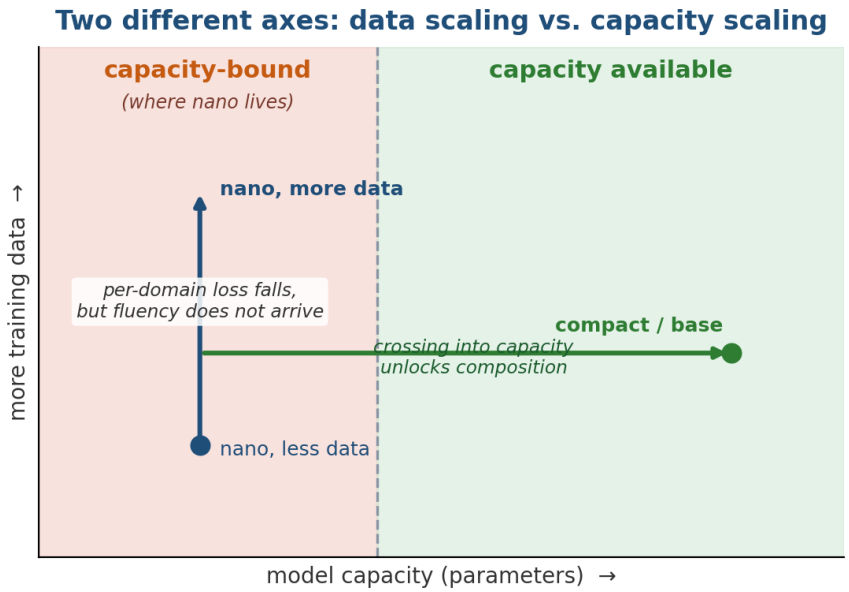


Figure 42 — The two axes are not interchangeable. At nano's parameter count, adding data moves the model up the vertical axis — lower per-domain loss — without crossing the vertical line into the regime where fluent composition becomes possible. Only added capacity crosses that line.

40.1 What more data does at a fixed size

The multi-domain result already on record is the anchor: the same nano model, trained across sixty-four domains totalling 70.7 MB, fell from **5.49 to 2.25 bits/byte** in held-out loss, learning five non-Latin languages from scratch at the byte level with no vocabulary change. Adding still more data to a model of this size continues to lower per-domain loss — the statistics of each domain are estimated more sharply — and it is the correct next experiment to run, because it is the one that isolates the data axis while holding capacity fixed.

But the architecture's own evidence predicts the ceiling this hits. At roughly 1–2 bits/byte the 5.4M-parameter model has learned each domain's statistics yet cannot compose them into sentences: a prompt for the author's name returns `Zahisal`, a mangled *Zahid*, rather than a word. That is the signature of a **capacity-bound** regime, not a data-starved one. More data sharpens the statistics the model already represents; it does not add the representational room needed to string them together.

40.1.1 The multi-domain result, measured

The table and figures below report the run in full. Training the 5.4M-parameter nano model for 25,000 steps across sixty-four domains — a nine-hour job on a laptop CPU — took the mean held-out loss from **5.49 to 2.25 bits/byte**, and a subsequent 20× author-recall fine-tune, EWC-protected, moved the author corpus from **3.32 to 2.59** while the 64-domain mean barely moved (1.539 → 1.516), confirming the fine-tune added a fact without erasing the breadth.

Measurement	Before	After	Change
64-domain mean held-out loss	5.49	2.25	−59%
Author corpus (20× fine-tune)	3.32	2.59	−22%
64-domain mean during fine-tune	1.539	1.516	held (EWC)
Best domain — Mathematics (K–12)	—	0.58	cleanest corpus
Worst large domain — Python / C++	—	2.24	code is hard
Non-Latin languages (5, from scratch)	—	1.31 – 1.81	no vocab change

Table 18 — The multi-domain training result in full: every figure measured on the trained nano weights, reported before and after.

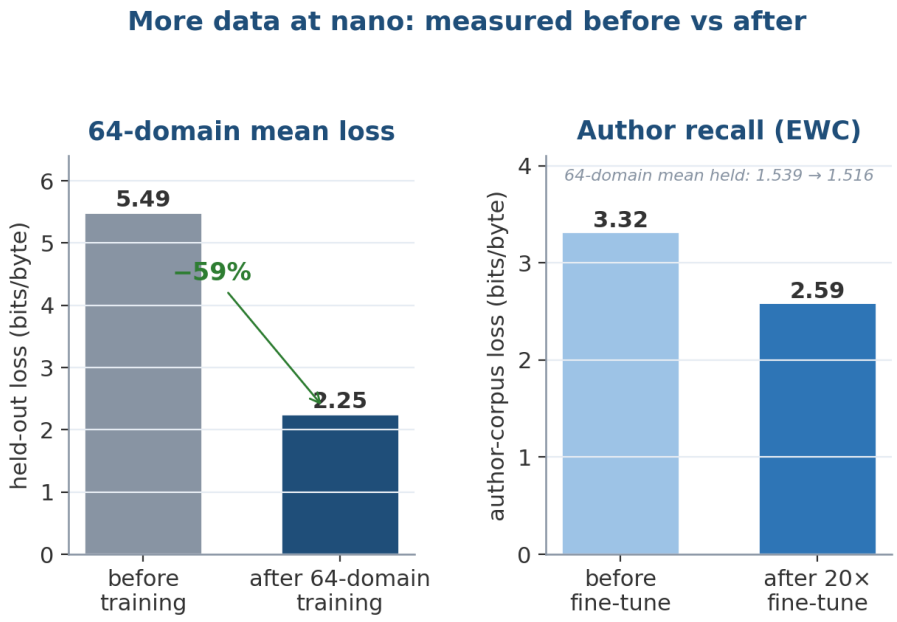


Figure 43 — The measured before/after. Left: mean held-out loss across all 64 domains falls 59% with training. Right: the author fine-tune lowers author-corpus loss while the 64-domain mean holds — targeted acquisition without measurable forgetting.

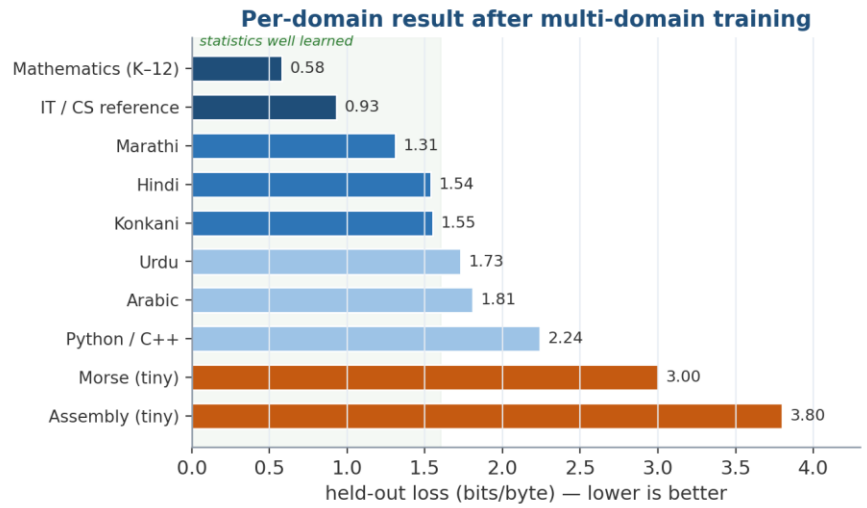


Figure 44 — Per-domain loss after training, lower is better. Clean, high-volume corpora (mathematics, reference text) are learned best; tiny corpora (Morse, assembly) and dense code are learned least. The five non-Latin languages were acquired from scratch at the byte level.

Every number here is real and reproducible on a CPU, and the qualitative result is reported beside them without softening: despite loss falling everywhere, **generation is still character-level fragments**. That pairing — a large, genuine drop in loss alongside output that is not yet words — is the whole point of this section. It is the direct, measured evidence that the remaining deficit is *capacity*, and it is what motivates the parameter axis below. A further data-scaling run at nano would extend this same table; under this paper's rule its loss numbers would again be reported alongside the generation result, so a better figure is never mistaken for fluency the size cannot yet deliver.

40.2 What more parameters are expected to do

The size ladder exists precisely for the step the data axis cannot take. The architecture is described once and instantiated at six scales; only **nano** has been trained end to end, while **small** and **compact** are built and forward/backward-verified but await a GPU or multi-day-CPU training job.

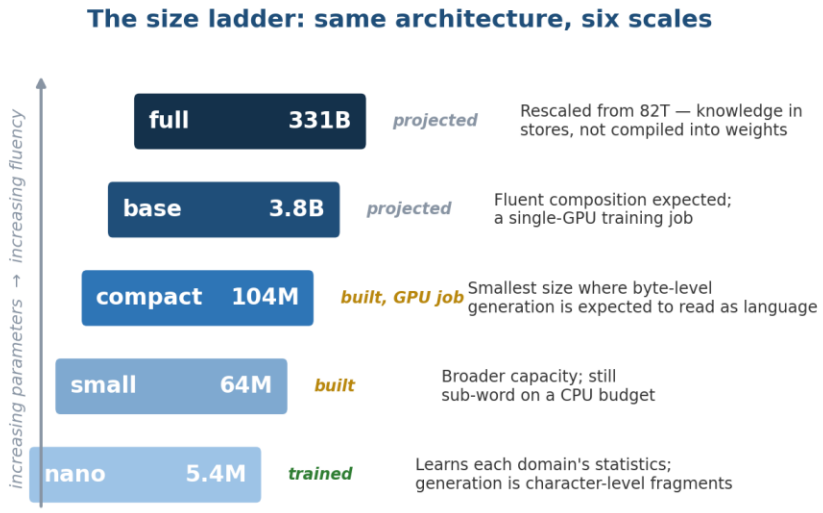


Figure 45 — The size ladder. The same architecture, corpus, method, and per-domain evaluation carry across every rung unchanged; only the parameter count and the required compute increase. Green is trained, amber is built, grey is projected.

The key projection is **compact**, at 104M parameters — roughly twenty times nano — described as the smallest size at which byte-level generation is expected to read as language rather than domain-flavoured character soup. **base**, at about 3.8B, is expected to compose fluently and is a single-GPU training job. The reason the expectation is credible is that nothing else has to change to test it: the corpus, the training method, and the per-domain evaluation all transfer unchanged; only the parameter count and the compute move. The claim being made is narrow and falsifiable — that the deficit is capacity, so adding capacity is what closes it — which is exactly the kind of claim the rest of the paper is built to state carefully.

Two structural features make added capacity cheaper than a raw parameter count suggests. First, the conscious pathway is **expert-routed**: only the top-two experts run per token, so total parameters grow far faster than the parameters active on any given input — the gap between total and active capacity is the point of the routing. Second, the design deliberately keeps lifelong knowledge in the episodic and semantic **stores** rather than compiling it into weights, where a store entry costs `d_model` floats against a weight matrix's `d_model²`. This is why the original **full** preset was rescaled from roughly 82T parameters to 331B: spending 82T parameters on weights would contradict the architecture's own claim that knowledge belongs in addressable memory, and cutting the widths while keeping the depth preserves the entire cognitive structure at a fraction of the parameters while raising memory capacity.

None of this is a measured result at scale, and the paper does not present it as one. Training **compact** and **base** are GPU-months of work that the CPU-only experiments here cannot stand in for, and the crossover into fluent generation is a prediction the size ladder is designed to test, not a number already in hand. What can be said with confidence is the conditional: *if* the deficit at nano is capacity rather than data or method — and the falling loss with persistent fragmentation is direct evidence that it is — *then* the larger presets are where the same architecture begins to generate the language its statistics already encode.

40.3 The cost of climbing the ladder

The projection above is honest only if the compute it requires is stated plainly, because the reason the larger rungs are untrained is not doubt about the method but the price of the hardware and time each demands. The figures below are the concrete training budget for each preset.

Scale	Params	Hardware	Training time	Cost
nano	5.4M	laptop CPU	9 hours (25k steps)	free — trained
compact	104M	budget GPU (RTX 3050)	24–48 hours	free (own GPU)
base	3.8B	high-end cloud GPU	~38 hours	~$300 (Colab)
full	331B	data-centre cluster	~1 month	millions

Table 19 — Training budget per scale. Only nano has been trained end to end; the rest are the compute each rung would require.

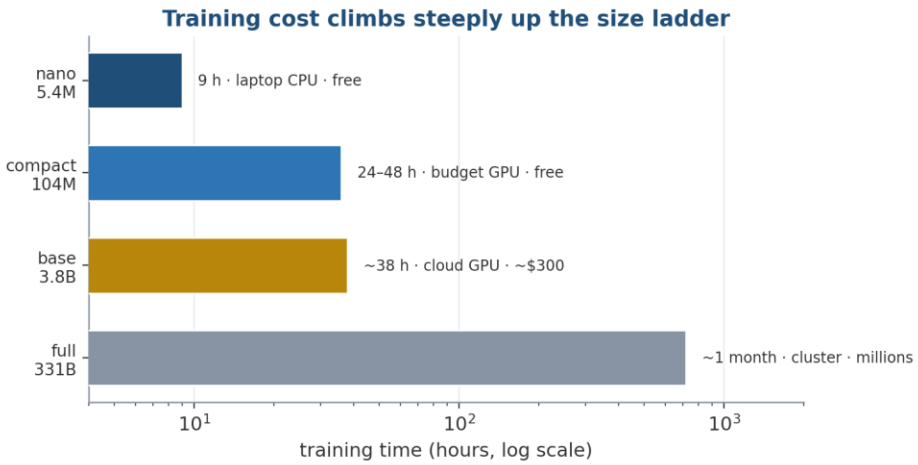


Figure 46 — Training time climbs steeply — and cost more steeply still — up the ladder. nano is a free overnight CPU job; compact is a weekend on a hobbyist GPU; base is a few hundred dollars of cloud time; full is a data-centre month. The step from nano to compact is the cheapest place to test the capacity hypothesis.

Read together with the results above, this table is the practical case for the roadmap. The jump from nano to **compact** — the rung where byte-level generation is expected to become legible — costs a day or two on a single budget GPU, not a data-centre. That is what makes the central claim testable in practice and not only in principle: the cheapest next experiment is also the one most likely to convert the measured drop in loss into readable output.

41. Strengths

- **Methodological discipline.** No component is described without a control, and every result — including the two losses and fourteen silent bugs — is reported. This is the single most transferable feature of the work.
- **Genuine gating.** The 2.16× speedup is a real wall-clock saving on the widest part of the model, validated by timing rather than by output equivalence.
- **Grounded epistemics with no oracle.** Source trust separates a good sensor from a broken one from identical priors, driven only by predictions resolving against observations.
- **Sybil resistance that scales the right way.** Behavioural clustering holds a protected axiom at 1.0000 against 600 puppets that otherwise drive it to 0.0001.
- **Structural forgetting protection.** The differentiability split keeps new facts out of the weights entirely, and EWC-plus-replay cuts measured forgetting by 74%.
- **A safety layer the learner cannot reach.** The non-learned safety controller is provably unreachable from online learning, and writing its tests surfaced two live hazards.
- **Reproducibility.** Every number reproduces on a CPU; 114 tests include a regression test for every catalogued defect.

42. Weaknesses

- **Scale.** Every experiment is at 5.4M parameters, on two tasks over 120 steps, with online stability over 400 updates. Convincing evidence needs many task sequences and 10^5–10^7 updates. This is the dominant limitation.
- **AMSA loses.** The architecture's own activation is beaten by SwiGLU on all three axes, and driving its adaptive state makes it marginally worse. It is retained as a default by authorial choice, clearly labelled.
- **The world model loses at one step.** A 16k-parameter linear layer predicts better than the 119k-parameter dynamics head, though the deficit does not propagate.

- **No robot, no simulator.** Nothing has been run on hardware or in simulation; BodyModel has never met a robot, and SelfCritique runs on a model too weak to critique much. Both are honest scaffolding.
- **Sybil resistance is a cost, not a guarantee.** A patient adversary varying its puppets' patterns, or one compromising genuinely independent sources, still gets through.
- **Triple extraction is out of scope.** The Reality Cortex reasons over (subject, relation, object) triples that something must supply; extracting reliable triples from raw perception is unsolved here.
- **Grounding covers a subset.** Only beliefs that imply a checkable prediction are grounded; a claim about something never observed stays anchored to its operator prior — correct behaviour, but partial coverage.
- **Spiking energy is a hardware argument.** On CPU or GPU, binary activations save no wall-clock time; the sparsity figures are a proxy for neuromorphic hardware.

43. Future Improvements

The remaining gaps are evidence, not architecture, and are compute- or hardware-bound rather than design-bound.

- **Scale the continual-learning test** to many task sequences and much longer horizons, and report stability–plasticity curves over a simulated lifetime.
- **Online learning at 10^5–10^7 updates,** to establish whether the trust region prevents collapse or merely slows drift — the two only diverge over a long run.
- **Multimodal and robotics benchmarks.** Vision, audio, navigation and manipulation are all wired and none are benchmarked; there is no simulator and no robot yet.
- **Memory benchmarks beyond retrieval,** including consolidation efficiency and forgetting curves over a long lifetime.
- **A larger AMSA experiment** across an order of magnitude in width, varying width and training length independently, to settle whether the crossover reverses again.
- **Perception-to-triple extraction,** closing the gap between raw sensor streams and the (subject, relation, object) facts the Reality Cortex consumes.
- **A real distillation run** against an open-weights teacher, to give the student genuine language competence before the cognitive machinery is exercised end to end.

44. Discussion

The most transferable contribution of this work may be methodological rather than architectural. A component that does nothing is observationally identical to one that works, until something is run without it. In this project that single principle recovered an EWC penalty contributing 10^{-4}, a rate limiter disabled by a shape comparison, a memory store collapsed to six percent of its contents, and an uncertainty head whose apparent anti-correlation was an artefact of measuring it off-distribution. None of these would have been visible in a loss curve, and all of them existed in code that ran without error and produced plausible output. The discipline of pairing every component with its control is cheap insurance against a whole class of failures that otherwise survive indefinitely.

The architectural thesis — that lifelong knowledge belongs in addressable, checkable memory rather than compiled into weights — is internally consistent across the design. It is why the memory hierarchy splits along differentiability, why the Reality Cortex sits on the knowledge path rather than the token path, why full was rescaled away from 82T parameters, and why online weight updates are kept rare and bounded. Whether the thesis is correct at useful scale is exactly the question

the compute-bound future work is meant to answer. The present contribution is to build the machinery and show, on small but honest experiments, which pieces earn their place and which do not.

45. Conclusion

BrainAI integrates spiking computation, a differentiability-split memory hierarchy, evidence-weighted belief maintenance, predictive world modelling, goal-directed executive control, hierarchical planning, end-to-end uncertainty, and a non-learned safety layer into one architecture. Of thirteen controlled comparisons, nine clearly favour the proposed component, one is mixed, and three favour the simpler alternative — and all are reported, including a fourteen-item catalogue of silent failures found only by running controls. The remaining work is not architectural. It is scale, embodiment, and the longer horizons under which the claims made here would either hold or fail.

> *Built by Mohammed Zahid Imityaz Wadiwale. Authorship is stamped into every module, config and checkpoint; BrainAI.load refuses to load weights whose attribution has been stripped or altered, while renaming passes cleanly.*

46. References

[1] Bellec, G., Salaj, D., Subramoney, A., Legenstein, R., & Maass, W. (2018). Long short-term memory and learning-to-learn in networks of spiking neurons. NeurIPS.

[2] Burda, Y., Edwards, H., Pathak, D., Storkey, A., Darrell, T., & Efros, A. (2019). Large-scale study of curiosity-driven learning. ICLR.

[3] Charikar, M. (2002). Similarity estimation techniques from rounding algorithms. STOC.

[4] Douceur, J. R. (2002). The Sybil attack. IPTPS.

[5] Erol, K., Hendler, J., & Nau, D. (1994). HTN planning: complexity and expressivity. AAAI.

[6] Fedus, W., Zoph, B., & Shazeer, N. (2022). Switch Transformers: scaling to trillion parameter models with simple and efficient sparsity. JMLR, 23.

[7] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. ICML.

[8] Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., & Davidson, J. (2019). Learning latent dynamics for planning from pixels. ICML.

[9] Hafner, D., Lillicrap, T., Ba, J., & Norouzi, M. (2020). Dream to control: learning behaviors by latent imagination. ICLR.

[10] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network. arXiv:1503.02531.

[11] Indyk, P., & Motwani, R. (1998). Approximate nearest neighbors: towards removing the curse of dimensionality. STOC.

[12] Kahneman, D. (2011). Thinking, Fast and Slow. Farrar, Straus and Giroux.

[13] Kim, Y., & Rush, A. M. (2016). Sequence-level knowledge distillation. EMNLP.

[14] Kirkpatrick, J., et al. (2017). Overcoming catastrophic forgetting in neural networks. PNAS, 114(13).

[15] Lv, Q., Josephson, W., Wang, Z., Charikar, M., & Li, K. (2007). Multi-probe LSH: efficient indexing for high-dimensional similarity search. VLDB.

[16] Maass, W. (1997). Networks of spiking neurons: the third generation of neural network models. Neural Networks, 10(9).

[17] Martins, A., & Astudillo, R. (2016). From softmax to sparsemax: a sparse model of attention and multi-label classification. ICML.

[18] McClelland, J. L., McNaughton, B. L., & O'Reilly, R. C. (1995). Why there are complementary learning systems in the hippocampus and neocortex. Psychological Review, 102(3).

[19] McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: the sequential learning problem. Psychology of Learning and Motivation, 24.

[20] Neftci, E. O., Mostafa, H., & Zenke, F. (2019). Surrogate gradient learning in spiking neural networks. IEEE Signal Processing Magazine, 36(6).

[21] Pathak, D., Agrawal, P., Efros, A. A., & Darrell, T. (2017). Curiosity-driven exploration by self-supervised prediction. ICML.

[22] Rolnick, D., Ahuja, A., Schwarz, J., Lillicrap, T., & Wayne, G. (2019). Experience replay for continual learning. NeurIPS.

[23] Shazeer, N., et al. (2017). Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. ICLR.

[24] Shazeer, N. (2020). GLU variants improve Transformer. arXiv:2002.05202.

[25] Vaswani, A., et al. (2017). Attention is all you need. NeurIPS.

[26] Zenke, F., Poole, B., & Ganguli, S. (2017). Continual learning through synaptic intelligence. ICML.

[27] Zhou, Z., et al. (2023). Spikformer: when spiking neural network meets Transformer. ICLR.

Appendix A. Every Block in the Architecture Diagram

This appendix walks the architecture figure (Figure 1/2) block by block, so no element is left unexplained.

A.1 Perception and processing pipeline (left, real-time)

- **Text / Image / Video / Audio / Touch.** The five input modalities. “Touch” in practice expands into a typed suite: IMU, LiDAR, ultrasonic, force/torque, tactile skin, and chemical (gas/olfactory) arrays.
- **Per-modality spiking encoders.** One encoder per modality, converting continuous input into spike trains and projecting into the shared model dimension. Binary forward pass; arctan surrogate on the backward pass.
- **Shared d_model spike bus.** The single representation every pathway reads. The commitment to genuine multimodality — fusion is a shared substrate, not a late concatenation.
- **REFLEX (shallow, always on).** One shallow, memory-free stack that always runs and produces a correct-enough answer immediately as a fallback.
- **SUBCONSCIOUS (always on, sparse).** Always-on sparse pathway that filters, predicts, and emits the salience score deciding whether the conscious pathway is worth running.
- **GATE.** Consumes salience. Soft during training (so the salience head learns), hard at inference (so skipping is a real wall-clock saving).
- **CONSCIOUS (wide, MoE, gated).** The wide, expert-routed pathway for reasoning, planning and memory integration. Runs only when salience clears the threshold; 53.7% of forward time.
- **Working Memory (read/write).** Differentiable attention-over-slots scratchpad, read and written by the conscious pathway; its contents are saved and restored across executive interrupts.
- **LTM interface (episodic + semantic recall).** The read path into the two non-differentiable stores, feeding recalled experiences and concepts into the conscious pathway.
- **ARBITER (mixes fast vs. deliberate).** Combines reflex and conscious outputs into the signal driving the two output heads. Formerly (mis)named the executive.
- **Language head.** Emits natural language from the arbiter's output.
- **Motor cortex.** Emits actuator commands from the arbiter's output.
- **SAFETY CONTROLLER (not learned, not reachable).** Non-learned, parameter-free gate between motor cortex and joints. Enforces envelope, constraints and interlock; provably unreachable from online learning (dashed red edge).
- **Joints (actuators / robot body).** The physical output. Receives only what the safety controller passes.

A.2 Control path (top-right): what should be running at all

- **EXECUTIVE.** Owns the goal stack, interrupts, task switching and sleep scheduling. Scores goals by priority × urgency × feasibility; schedules sleep from four pressures with idleness as a multiplier.
- **PLANNER.** HTN-style decomposition of a goal into ordered subgoals, recursively to primitive leaves. Scores candidate plans by imagined rollout; distinguishes retry / remethod / failed.
- **WORLD MODEL (imagine / simulate).** Rolls candidate plans forward in latent space without touching the environment, so a bad plan costs matmuls rather than hardware. Also supplies calibrated rollout confidence.
- **ACTION (selected plan).** The chosen plan, emitted toward the motor cortex through the arbiter.

A.3 Knowledge path (bottom-right): offline, not per-token

- **EPISODIC MEMORY.** Importance-ranked experiences in a vector store. The entry point to the knowledge path and the source of replay material.
- **REALITY CORTEX.** Evidence pooling in log-odds, source-trust maintenance, contradiction detection, collusion clustering, and promotion. Separates important from true.
- **SEMANTIC MEMORY.** Concepts and the edges between them, populated by promoted beliefs. A vector store plus a graph.
- **SLEEP (consolidation).** Replays raw tokens with the EWC penalty active, merges near-duplicates, decays the unimportant, re-anchors, and promotes beliefs toward the weights.
- **WEIGHTS (neural parameters).** The slow store. Only importance-gated consolidation ever writes here, which is what protects old skills.
- **WORLD MODEL (prediction error).** Feeds the knowledge path: prediction error resolving against observation is what grounds source trust with no oracle.

A.4 The two annotations on the diagram

- **“Reflex answers immediately; subconscious scores salience.”** The dual-process reading: fast pathways act while the slow one is gated on whether it is worth the cost.
- **“Confidence is carried alongside every stage … so the safety controller can refuse to act on a decision the pipeline was never sure about.”** The end-to-end uncertainty thread, composed multiplicatively with a floor, terminating in a veto capability at the safety controller.
- **“Not learned, not reachable by online learning” (dashed red).** The structural guarantee that the policy which rewrites its own weights cannot widen its own operating envelope.

Appendix B. Repository Layout and Reproducibility

The implementation is organised into modules, one per subsystem described above.

```
brainai/
  identity.py       attribution, renaming, tamper checking
  config.py         scale presets, section specs
  activation.py     AMSA, GatedAMSA
  neurons.py        ALIF, surrogate gradients, spiking LSTM
  attention.py      spike attention, sparsemax, RMSNorm
  blocks.py         spiking blocks, MoE routing, expert growth
  encoders.py       text / image / video / audio / touch
  memory.py         the five memories, hippocampal controller, LSH index
  pathways.py       reflex, subconscious, conscious, arbiter, motor
  sensors.py        typed sensors incl. olfaction; rate/latency/drift fusion
  actuators.py      DC / stepper / servo / hydraulic / pneumatic envelopes
  world.py          dynamics, imagination, curiosity, body model, ledger
  reality.py        evidence, source trust, collusion, belief promotion
  executive.py      goals, interrupts, task switching, sleep scheduling
  planner.py        HTN decomposition, imagined scoring, replanning
  uncertainty.py    per-stage confidence and its composition
  critique.py       self-critique loop, internal expert debate
  safety.py         joint envelope, constraints, e-stop  (not learned)
  plasticity.py     Fisher, EWC, online updates, growth
  consolidation.py  replay buffer, sleep
  tokenizer.py      byte-level
```

```
  data.py          corpora, evaluation
  model.py         the assembled model
```

Checkpoints are ~3× the model size because the plasticity controller stores a Fisher estimate, an anchor and a plasticity value per weight — the cost of the continual-learning machinery, and why a 5M-parameter nano checkpoint is ~89 MB. Every result reproduces on a CPU; the test suite comprises 114 tests, including a regression test for every silent failure in Section 32.

B.1 Commands to reproduce every result

```
pip install torch
python train.py --steps 400              # pretrain (CPU, ~10 min)
python continual.py --steps 120          # the anti-forgetting measurement
python demo.py                           # every capability, end to end
python -m pytest tests -q                # 114 tests

python benchmarks/cognition_bench.py     # world model, curiosity, grounding, planner
python benchmarks/worldmodel_jobs.py     # the world model's actual jobs
python benchmarks/amsa_continual.py --sweep
python benchmarks/belief_bench.py        # Reality Cortex vs a naive baseline
python benchmarks/gating_bench.py        # is the conscious pathway really skipped?
python benchmarks/stability_bench.py --control   # 400 online updates
```

Appendix C. Glossary of Key Terms

AMSA — Adaptive Memory Spike Activation — an activation whose threshold and gain are driven by per-channel plasticity, confidence and memory state.

ALIF — Adaptive Leaky-Integrate-and-Fire — a spiking neuron whose firing threshold rises after each spike and decays over tau_adapt.

Arbiter — The module that mixes fast (reflex) and deliberate (conscious) outputs.

EWC — Elastic Weight Consolidation — a penalty pulling weights toward an anchor in proportion to their Fisher importance.

Fisher — The diagonal Fisher information, used as a per-weight importance estimate for plasticity gating and EWC.

HTN — Hierarchical Task Network — the planning style in which a method library maps goals to ordered subgoals recursively.

MoE — Mixture-of-Experts — the expert-routed structure of the conscious pathway; only the top-k experts run per token.

Plasticity — Per-parameter 1 − normalised Fisher; scales online gradients so important weights barely move.

Prediction ledger — The structure that lets a belief stake a claim on a future observation, resolving it to ground source trust.

Reality Cortex — The epistemic layer that separates important from true via evidence pooling, discounts, and collusion detection.

Salience — The subconscious pathway's score for whether the conscious pathway is worth running.

Trust region — The bound on an online update's magnitude, a fraction of the parameter norm.

— End of paper —

Built by Mohammed Zahid Imityaz Wadiwale · zahid.wadiwale1234@gmail.com

Appendix D. Complete Mathematical Reference

This appendix collects every equation in the implementation, grouped by source module and typeset in full. It is exhaustive by design: the prose sections above explain *why* each mechanism exists, whereas this reference is meant to be read alongside the code so that no formula lives only in a comment. Symbols follow the conventions established in the body — σ is the logistic sigmoid, ⊙ is elementwise product, overlines denote running or batch means, and hatted quantities are model predictions.

D.1 AMSA — activation.py

The adaptive-memory spike activation. The threshold, gate sharpness, and amplifier are learned per channel; the memory, plasticity, and confidence buffers **m, p, c** are written by the plasticity controller, never by gradient descent. The amplifier is evaluated as `-expm1(-β|z|)` for accuracy at small argument.

$$\theta = \theta_0 + \lambda p \qquad \alpha = \mathrm{softplus}(\alpha_0 + \delta c) + \epsilon \qquad \beta = \mathrm{softplus}(\beta_0) + \epsilon$$

$$z = x - \theta \qquad f(x) = z\,\sigma(\alpha z)\,(1 - e^{-\beta|z|})\,(1 + \gamma m)$$

$$a \leftarrow 0.99\,a + 0.01\,\overline{|f(x)|}$$

$$\mathrm{GatedAMSA}(x) = W_2(\mathrm{AMSA}(W_1 x) \odot \sigma(W_g x))$$

$$\mathrm{SwiGLU}(x) = W_2(\mathrm{SiLU}(W_g x) \odot W_1 x)$$

activation.py — AMSA, its activity tracker, and the gated variants against SwiGLU.

D.2 ALIF neurons — neurons.py

The adaptive leaky integrate-and-fire neuron. Decays are parameterised in logit space and initialised from time constants; the arctan surrogate is used on the backward pass only. Because `a[0] = s[0] = 0`, the adaptation term multiplies zero until t = 2, which is the formal reason at least three timesteps are required.

$$\lambda_m = \sigma(\ell_m), \qquad \rho = \sigma(\ell_a)$$

$$v[t] = \lambda_m \, v[t-1] \, (1 - s[t-1]) + (1 - \lambda_m) \, I[t]$$

$$a[t] = \rho \, a[t-1] + s[t-1] \qquad \vartheta[t] = v_{th} + \kappa_a \, a[t]$$

$$s[t] = H(v[t] - \vartheta[t]) \qquad \frac{\partial s}{\partial v} \approx \frac{\varsigma}{2\,(1 + (\frac{\pi}{2}\varsigma v)^2)}$$

neurons.py — membrane update, adaptation, threshold, spike, and surrogate derivative.

D.3 Attention — attention.py

Scaled dot-product scores, RMSNorm, and the sparsemax projection onto the simplex. Sparsemax yields exact zeros, which suits the sparse spike representation better than a dense softmax.

$$\text{scores} = \frac{Q\,K^{\top}}{\sqrt{d_h}} \qquad \text{RMSNorm}(x) = \frac{x}{\sqrt{\overline{x^2} + \epsilon}} \odot g$$

$$\tau = \frac{\sum_{j \le k} z_{(j)} - 1}{k_{max}} \qquad p = [\, z - \tau \,]_{+} \quad \text{(sparsemax)}$$

attention.py — scores, RMSNorm, and the sparsemax threshold.

D.4 Spiking blocks — blocks.py

Rate coding averages the activation over the spike window; pre-norm residual connections wrap attention and the feed-forward block; the mixture-of-experts router selects and renormalises the top-k experts, with a load-balancing auxiliary loss.

$$\text{out} = \frac{1}{T} \sum_{t=1}^{T} f(\text{ALIF}_t(x))$$

$$x \leftarrow x + \text{Attn}(\text{RMSNorm}(x)) \qquad x \leftarrow x + \text{FFN}(\text{RMSNorm}(x))$$

$$p = \text{softmax}(W_r s) \qquad w_e = v_e / \sum v \qquad y = \sum_{e \,\in\, top\,k} w_e \, E_e(s)$$

$$\mathcal{L}_{aux} = n_{active} \sum_{e} f_e \, \bar{P}_e$$

blocks.py — rate coding, residual structure, MoE routing, and the load-balance term.

D.5 Plasticity, EWC, and online updates — plasticity.py

The Fisher information is tracked by EMA and normalised by its 95th percentile — not its maximum, which would protect almost nothing given the heavy tail. The EWC penalty is self-calibrated so a 1% relative drift costs exactly `λ/2` nats, and the online step masks the gradient by plasticity, clips relative to the weight scale, and rolls back if the loss rose.

$$F \leftarrow 0.95\,F + 0.05\,g^2$$

$$\hat{F} = \min(F / Q_{0.95}(F),\ 1) \qquad \pi = \mathrm{clamp}(1 - \hat{F},\ \pi_{min},\ 1)$$

$$\mathcal{L}_{EWC} = \tfrac{1}{2}\lambda\, s \sum_i F_i(\theta_i - \theta_i^*)^2$$

$$s = 1/\max(\mathrm{ref}, \epsilon) \qquad \mathrm{ref} = \sum_i F_i(0.01\,\theta_i^*)^2$$

$$\tilde{g} = g \odot \pi \qquad \theta \leftarrow \theta - \mathrm{lr} \cdot \mathrm{clip} \cdot \tilde{g}$$

plasticity.py — Fisher EMA, plasticity mask, calibrated EWC, and the bounded update.

D.6 World model — world.py

The dynamics head predicts the next latent; the error head predicts its own expected error, giving a confidence `e^(-ê)`. Curiosity is surprise damped by that expected error — the noisy-TV defence — and rollout confidence compounds multiplicatively over the horizon.

$$\hat{s}' = \mathrm{RMSNorm}(s + \mathrm{MLP}([\,s;\ W_a\,\mathrm{act}\,]))$$

$$\hat{e} = \mathrm{softplus}(\mathrm{MLP}_e(\cdot)) \qquad \mathrm{conf} = e^{-\hat{e}}$$

$$\mathcal{L} = \mathrm{MSE}(\hat{s}', s') + 0.1\,\mathrm{MSE}(\hat{e},\ \mathrm{err})$$

$$\mathrm{surprise} = \mathrm{clamp}((\mathrm{err} - \mu)/\sigma,\ -8,\ 8) \qquad C = \prod_t \mathrm{conf}_t$$

$$r = \mathrm{clamp}\big(\sigma(\mathrm{surprise}) \cdot e^{-\hat{e}} \cdot \nu,\ 0,\ 1\big)$$

world.py — prediction, calibrated error head, loss, surprise, rollout confidence, curiosity.

D.7 Reality Cortex — reality.py

Source trust is a Beta posterior mean; credibility shrinks toward the prior for thin evidence; within-source contributions are damped, colluding clusters collapse to a single vote, and independent sources are discounted by 1/√k. Protected axioms carry a large prior log-odds that decays slowly.

$$t = \frac{\pi_0 S + n_+}{\pi_0 S + n_+ + (1-\pi_0) S + n_-} \qquad c = \frac{n_{eff}}{n_{eff} + \kappa}, \quad \kappa = 4$$

$$C_{src} = \sum_i \frac{\text{sign}_i \cdot \log(t/(1-t)) \cdot c}{1 + n_i} \qquad \ell = \ell_{prior} + \sum_k \frac{C_{(k)}}{\sqrt{k}}$$

$$\text{conf} = \sigma\Big(\ell - \max_{j \neq i} \ell_j\Big) \qquad \text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|} \geq 0.85$$

reality.py — trust, shrinkage, pooling, cluster collapse, and the collusion test.

D.8 Memory — memory.py

Retrieval similarity and novelty; the eviction value combining importance, hits, and recency; an adaptive merge threshold set from the off-diagonal statistics; the differentiable working-memory read/write; mean-centred addressing; and the LSH bucket hash.

$$\text{sim} = \hat{q}\,\hat{K}^\top \qquad \text{novelty} = \frac{1 - \max(\text{sim})}{2}$$

$$V = \text{importance} + 0.3 \log(1 + \text{hits}) + 0.2\,\text{recency}$$

$$\tau = \min(0.999,\ \mu_{od} + 4\,\sigma_{od}) \qquad r = \text{softmax}\Big(\frac{QM^\top}{\sqrt{d}}\Big) M$$

$$k = W_k(\text{summary} - \mu) \qquad \mu \leftarrow 0.99\mu + 0.01\,\overline{\text{summary}}$$

$$h_t(v) = \sum_b 2^b\, \mathbb{1}[\langle H_{t,b},\ v \rangle > 0]$$

memory.py — similarity, eviction, adaptive merge, working memory, addressing, LSH.

D.9 Pathways and executive — pathways.py, executive.py

The arbiter's confidence-weighted mix and final logits; the urgency curve — bounded below by its baseline so a deadline can never *reduce* urgency; the multiplicative goal score; and the four-term sleep-pressure sum.

$$w = \sigma(W[r;\, d;\, \text{conf}]) \qquad \text{mixed} = w\, r + (1 - w)\, d$$

$$\text{logits} = w \cdot \text{reflex}(r) + (1 - w) \cdot \text{lm}(\text{mixed})$$

$$u = u_0 + (1 - u_0)\tfrac{1}{1 + 0.1\,\Delta t}, \quad u_0 = 0.5 \qquad \text{score} = \text{priority} \times u \times \text{feasibility}$$

$$P = \min(1, \text{fill}) + \min(1, \tfrac{\text{claims}}{32}) + \min(1, \tfrac{\text{drift}}{0.5}) + \min(1, \tfrac{\text{ticks}}{512})$$

pathways.py / executive.py — arbiter mix, urgency, goal score, sleep pressure.

D.10 Uncertainty — uncertainty.py

Three confidence estimators — entropy, margin, and spike-rate — and their composition as a floored geometric mean, which stays informative where a raw product would collapse. Expert disagreement is one minus mean pairwise cosine.

$$\text{conf}_H = 1 - \tfrac{H(p)}{\log n} \qquad \text{conf}_{margin} = \sigma(z_{(1)} - z_{(2)}) \qquad \text{conf}_{spike} = e^{-|r - r_0|/\tau}$$

$$\text{overall} = \Big(\prod_i \max(c_i, 0.05)\Big)^{1/n} \qquad D = \text{clamp}\big(\tfrac{1 - \bar{c}}{2}, 0, 1\big)$$

uncertainty.py — entropy/margin/spike confidences, composition, disagreement.

D.11 Safety and hardware — safety.py, sensors.py, actuators.py

The non-learned safety clamp that bounds both absolute command and per-step change; stepper quantisation; sensor confidence under noise and drift with a saturation floor; and the staleness and fused-rate rules. None of this is reachable by the online learner.

$$\text{safe} = \text{last} + \text{clamp}\big(\text{clamp}(\text{cmd}, \ell, u) - \text{last},\ -\Delta_{max},\ +\Delta_{max}\big)$$

$$q = \text{round}(\text{cmd}/\delta) \cdot \delta \qquad \delta = 2\,\text{travel}/N_{steps}$$

$$\text{conf} = \tfrac{1}{1 + 8\,\text{noise}} \cdot \tfrac{1}{1 + \text{drift} \cdot t_h} \qquad t_{ms} > 2/f + \text{latency} \qquad f_{fused} = \min_i f_i$$

safety.py / sensors.py / actuators.py — safe command, quantisation, sensor confidence, staleness.

D.12 Losses and sizing

The language loss with its auxiliary term, the sleep loss adding identity preservation and the EWC penalty, and the closed-form parameter estimate used by the scale presets.

$$\mathcal{L}_{LM} = \mathrm{CE}(\text{shifted}) + 0.01\,\mathcal{L}_{aux}$$

$$\mathcal{L}_{sleep} = \mathcal{L}_{LM} + 0.1\,\mathrm{MSE}(\hat{\imath}, \imath^{*}) + \mathcal{L}_{EWC}$$

$$\text{params} \approx \sum_{layers} (4 + 12\,n_{ffn})\,w^2 + 2wd + 3Vd + S_{max}\,d$$

training — language loss, sleep loss, and the parameter-count estimate.

Reading guide. The two mechanisms that *lose* to their controls in the measurements — AMSA (D.1) and the world-model dynamics head (D.6) — are kept here in full precisely because the paper's methodological claim is that every equation was measured against a simpler alternative, and the ones that failed are reported rather than removed.